

11

VISÃO QUE ENXERGA: SEGMENTAÇÃO E EXTRAÇÃO DE FEATURES EM IMAGENS DIGITAIS

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Diagrama de fluxo do pipeline de Visão Computacional 8
Figura 2 - Diferentes métodos de segmentação aplicados à mesma imagem 12
Figura 3 - Limiarização com diferentes limiares T 15
Figura 4 - Limiarização com Otsu 16
Figura 5 - Comparação: limiar global, Otsu e limiar adaptativo 17
Figura 6 - *Region Growing* aplicada a exame de imagem do cérebro humano 20
Figura 7 - Segmentação *Split and Merge* aplicado ao diagnóstico de melanoma 21
Figura 8 - Segmentação Mean-shift aplicada a tomografia cerebral. Original (esquerda) e segmentada (direita) 22
Figura 9 - Sobel aplicada a tomografia cerebral. Original (esquerda) e Sobel (direita) 24
Figura 10 - Canny aplicado a tomografia cerebral. Original (esquerda) e Canny (direita) 26
Figura 11 - Canny aplicado a tomografia cerebral. Original (esquerda) e Canny + detecção de contornos (direita) 27
Figura 12 - Features globais (direita) e locais (esquerda) 29
Figura 13 - O mesmo objeto é capturado em diferentes escalas, orientações e condições de iluminação, com keypoints detectados de forma consistente 30
Figura 14 - Pipeline genérico de um sistema de Visão Computacional 31
Figura 15 - Resultado da aplicação do SIFT 33
Figura 16 - Resultado da aplicação do SURF 34
Figura 17 - Resultado da aplicação do ORB 35
Figura 18 - Resultado ORB (cima) *versus* SIFT (embaixo) 37
Figura 19 - Histograma RBG e HSV 39
Figura 20 - GLCM 40
Figura 21 - LBP 41
Figura 22 - Selective Search 44
Figura 23 - R-CNN 45

LISTA DE QUADROS

Quadro 1 - Principais diferenças entre SIFT, SURF e ORB.....35

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Filtros de Limiarização 18
Código-fonte 2 - Mean shift no opencv 22
Código-fonte 3 - Aplicação do Sobel em tomografia do cérebro 24
Código-fonte 4 - Aplicação do Canny em tomografia do cérebro 25
Código-fonte 5 - Aplicação do Canny + detecção de contornos em tomografia do cérebro 27
Código-fonte 6 - SIFT *versus* ORB 36
Código-fonte 7 - Métodos de extração de features globais 42

FIAP

SUMÁRIO

1 VISÃO QUE ENXERGA: SEGMENTAÇÃO E EXTRAÇÃO DE FEATURES EM IMAGENS DIGITAIS 7
1.1 Introdução 7

2 SEGMENTAÇÃO DE IMAGENS 10

3 MÉTODOS DE LIMIARIZAÇÃO (THRESHOLDING) 14
3.1 Limiarização global 14
3.2 Método de Otsu 15
3.3 Limiarização adaptativa 16
3.4 Pontos fortes e fracos 17
3.5 Exemplos de Código com OpenCV 18

4 SEGMENTAÇÃO BASEADA EM REGIÕES 19
4.1 Crescimento de regiões (Region growing) 19
4.2 Divisão e fusão (region splitting and merging) 20
4.3 Segmentação por Mean-Shift 21

5 SEGMENTAÇÃO BASEADA EM BORDAS (EDGE-BASED SEGMENTATION) 23
5.1 Operadores de gradiente 23
5.2 Detector de Bordas de Canny 25
5.3 Detecção de Contornos 26

6 FUNDAMENTOS DA EXTRAÇÃO DE CARACTERÍSTICAS (FEATURE EXTRACTION BASICS) 28
6.1 Características locais versus características globais 28
6.2 Propriedades desejáveis de características 29
6.3 Descritores *versus* valores brutos de pixels 30
6.4 Fluxo no pipeline de Visão Computacional 31

7 DESCRITORES LOCAIS DE CARACTERÍSTICAS (LOCAL FEATURE DESCRIPTORS) 32
7.1 SIFT (Scale-Invariant Feature Transform) 32
7.2 SURF (Speeded Up Robust Features) 33
7.3 ORB (Oriented FAST and Rotated BRIEF) 34
7.4 Comparação entre SIFT, SURF e ORB 35
7.5 Implementação em OpenCV 36

8 DESCRITORES GLOBAIS DE CARACTERÍSTICAS (GLOBAL FEATURE DESCRIPTORS) 38
8.1 Histogramas de cores 38
8.2 Descritores de textura 39
8.2.1 Haralick Features e a Matriz de Coocorrência de Níveis de Cinza (GLCM) 40
8.2.2 LBP (Local Binary Patterns) 40
8.3 Implementação em Python (OpenCV e Scikit-Image) 42

9 OBJECT PROPOSALS E IDEIAS DE DETECÇÃO BASEADA EM REGIÕES 43
9.1 O Algoritmo Selective Search 43
9.2 A Família R-CNN: Uma visão conceitual 44
9.3 Conexão com aplicações do mundo real 45

CONCLUSÃO 46

FINAL

REFERÊNCIAS.....47

FIAP

1 VISÃO QUE ENXERGA: SEGMENTAÇÃO E EXTRAÇÃO DE FEATURES EM IMAGENS DIGITAIS

1.1 Introdução

A segmentação de imagens é um dos passos mais importantes dentro de um sistema de Visão Computacional, pois é nesse momento que se estabelece, de maneira explícita, quais partes de uma imagem são relevantes para a tarefa que se deseja realizar. Em termos simples, segmentar significa dividir uma imagem em regiões ou objetos com propriedades semelhantes, de forma que possamos isolar o que interessa e descartar o que é irrelevante para a análise. Essa etapa atua como um filtro inteligente: enquanto o pré-processamento cuida de remover ruídos e ajustar a qualidade visual, a segmentação separa o conteúdo da imagem em blocos significativos para interpretação posterior.

No fluxo clássico de um pipeline de Visão Computacional, a segmentação surge após a aquisição e o pré-processamento da imagem, mas antes da etapa de extração de características e da classificação. Uma imagem digital, sem qualquer processamento, é apenas uma matriz de valores numéricos que representam intensidades ou cores. Para que um algoritmo possa reconhecer padrões, identificar objetos ou tomar decisões, é necessário extrair dessa matriz apenas a porção que contém informação relevante. É exatamente aqui que a segmentação cumpre seu papel, recortando, por assim dizer, a “área de interesse” que será analisada.

A importância desse processo se torna mais clara quando se pensa em aplicações reais. Em um exame de ressonância magnética, por exemplo, a imagem bruta contém não apenas o tumor, mas também tecidos, ossos e outras estruturas irrelevantes para a análise específica. Um radiologista, ao interpretar o exame, foca no tumor, ignorando o restante do conteúdo. O mesmo princípio vale para algoritmos: ao aplicar segmentação, isolamos o tumor, permitindo que, na etapa seguinte, possamos extrair informações precisas — como forma, tamanho, textura — e alimentar um classificador que determine se a lesão é benigna ou maligna.

Esse processo de passar de uma imagem bruta para uma representação mais “condensada” e informativa é complementado pela extração de características.

Enquanto a segmentação delimita o objeto, a extração de características descreve esse objeto numericamente. As características, ou *features*, podem representar cor, forma, bordas, padrões de textura, entre outros aspectos. É como descrever uma pessoa não pela sua foto completa, mas por atributos como altura, peso, cor dos olhos e estilo de cabelo. Essas informações, convertidas em números, são essenciais para que os algoritmos de aprendizado de máquina consigam diferenciar, por exemplo, uma maçã de uma laranja, mesmo que ambas tenham formatos semelhantes.

Em sistemas de classificação, a qualidade da segmentação influencia diretamente o desempenho do modelo. Uma segmentação imprecisa — que inclua partes irrelevantes ou que exclua partes importantes do objeto — pode levar a descrições distorcidas e classificações erradas. Por outro lado, uma segmentação bem executada simplifica o problema, reduz a dimensionalidade dos dados e melhora significativamente a precisão do reconhecimento.

Os exemplos de uso dessa combinação de segmentação e extração de características são vastos e cada vez mais presentes no cotidiano. Nos carros autônomos, os algoritmos segmentam a cena para identificar a pista, as calçadas, outros veículos e pedestres, possibilitando a tomada de decisão em tempo real para evitar acidentes. Na medicina, ferramentas de Visão Computacional auxiliam médicos a segmentar automaticamente órgãos e estruturas anatômicas, acelerando diagnósticos e planejamentos cirúrgicos. No varejo, câmeras acopladas a sistemas inteligentes identificam e classificam produtos nas prateleiras, atualizando inventários sem a necessidade de intervenção humana.

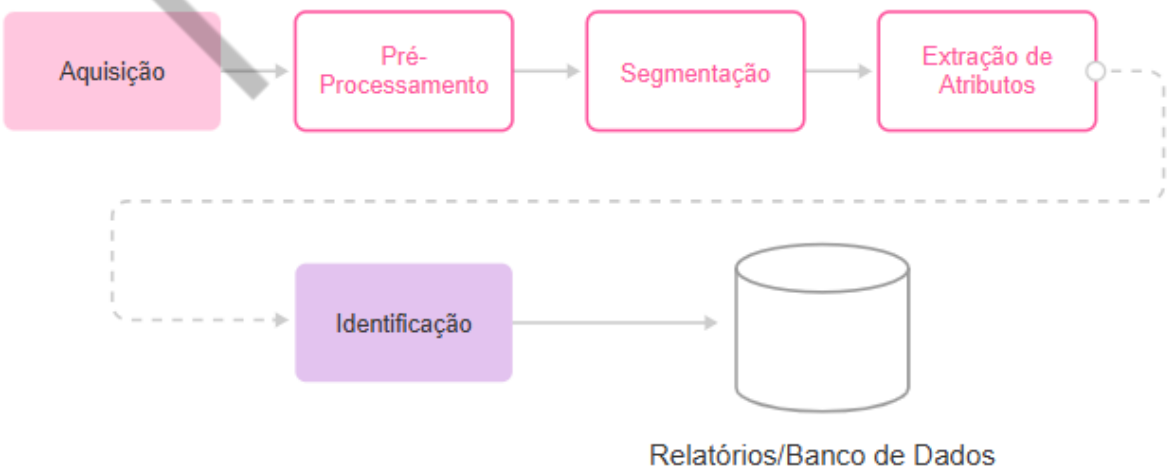


Figura 1 - Diagrama de fluxo do pipeline de Visão Computacional
Fonte: ResearchGate (2015), adaptado por FIAP (2025)

Em *Digital Image Processing* de Gonzalez e Woods (2018), a segmentação é descrita como o processo que “divide uma imagem em suas partes ou objetos constituintes”, sendo considerada uma das etapas mais difíceis, mas também mais decisivas, de todo o processamento. Já em *Computer Vision: Algorithms and Applications*, Szeliski (2022) destaca que a segmentação, aliada à extração de características robustas, é o que permite a um sistema interpretar de forma confiável o conteúdo de uma cena.

Ao longo deste capítulo, exploraremos métodos fundamentais e avançados para segmentar imagens e extrair características relevantes, sempre com foco no entendimento prático e em aplicações reais. A ideia é que, ao final, o leitor compreenda não apenas a teoria por trás desses processos, mas também saiba aplicá-los para resolver problemas concretos em Visão Computacional.

FIAP

2 SEGMENTAÇÃO DE IMAGENS

Para compreender a segmentação de imagens de maneira sólida, é necessário voltar alguns passos e entender a base matemática e conceitual sobre a qual essas técnicas se apoiam. Segmentar não é apenas “recortar” um objeto na imagem, mas identificar, com base em critérios objetivos, quais pixels pertencem a uma determinada região de interesse. Esses critérios se apoiam em conceitos fundamentais como valor de intensidade, conectividade, medidas de similaridade e dissimilaridade, bem como na escolha entre métodos supervisionados ou não supervisionados. Além disso, a qualidade da segmentação precisa ser mensurada de forma consistente, de modo que possamos comparar diferentes algoritmos ou avaliar se o resultado atende ao objetivo proposto.

No nível mais básico, uma imagem digital é composta por pixels. Cada pixel é uma unidade mínima de informação visual que possui um valor de intensidade (no caso de imagens em tons de cinza) ou um conjunto de valores que representam cores (em imagens RGB, por exemplo). Esses valores numéricos são a base sobre a qual qualquer decisão de segmentação será tomada. Em uma imagem de tom de cinza de 8 bits, por exemplo, cada pixel pode assumir valores entre 0 (preto) e 255 (branco), com diferentes gradações intermediárias.

Um conceito intimamente ligado à segmentação é o de conectividade entre pixels. Em imagens bidimensionais, dois pixels são considerados conectados se atendem a dois critérios: primeiro, se seus valores de intensidade ou cor são suficientemente próximos considerando uma medida definida; segundo, se ocupam posições vizinhas no espaço da imagem. Existem duas formas comuns de definir essa vizinhança: conectividade de 4 e conectividade de 8. Na conectividade de 4, um pixel é considerado vizinho apenas dos pixels que estão imediatamente acima, abaixo, à esquerda e à direita. Na conectividade de 8, incluem-se também os pixels que estão nas diagonais. Essa diferença, aparentemente simples, pode mudar completamente a forma como as regiões são agrupadas, influenciando diretamente o resultado da segmentação.

Outro ponto crucial é a definição de similaridade e dissimilaridade. A segmentação, de forma geral, procura agrupar pixels que são semelhantes segundo

algum critério. Essa similaridade pode ser medida pela diferença de intensidade, pela distância no espaço de cores, por padrões de textura ou mesmo por características mais complexas obtidas a partir de modelos de aprendizado profundo. Um exemplo prático está no agrupamento de áreas de uma fotografia aérea: para identificar zonas de vegetação, um algoritmo pode considerar pixels com tonalidades de verde semelhantes como pertencentes a uma mesma região, enquanto áreas urbanas terão valores muito diferentes, indicando dissimilaridade.

Quando falamos sobre métodos supervisionados e não supervisionados de segmentação, nos referimos à forma como o algoritmo aprende a identificar as regiões de interesse. Na segmentação supervisionada, o sistema é treinado com imagens previamente rotuladas, nas quais cada pixel já foi marcado como pertencente a uma classe específica (por exemplo, “céu”, “árvore” ou “edifício”). Esse tipo de abordagem depende de um conjunto de dados de treinamento e, portanto, é tão bom quanto a qualidade e a representatividade desse conjunto. Já na segmentação não supervisionada, o algoritmo não recebe rótulos previamente definidos; ele precisa descobrir padrões e agrupamentos por conta própria, geralmente com base na análise estatística das propriedades dos pixels — como ocorre nos métodos de clustering (exemplo: k-means, mean-shift).

Um aspecto frequentemente negligenciado, mas de extrema importância, é a avaliação da qualidade da segmentação. Para saber se um método funcionou bem, não basta observar visualmente o resultado; é necessário quantificar a qualidade da segmentação. Duas métricas amplamente utilizadas são o **IoU** (Intersection over Union) e o **Coeficiente de Dice**.

O **IoU** mede a proporção da interseção entre a região segmentada e a região de referência (também chamada de *ground truth*) sobre a união dessas regiões. Por exemplo, se a interseção cobre 80 pixels e a união cobre 100 pixels, o IoU será 0,8, indicando um alto grau de sobreposição. O **Coeficiente de Dice** é uma métrica semelhante, mas dá mais peso à interseção, sendo definido como o dobro da interseção dividido pela soma das áreas das duas regiões. Valores próximos de 1 em ambas as métricas indicam segmentações de alta qualidade, enquanto valores baixos sinalizam divergência entre o resultado obtido e o esperado.

Na prática, a escolha da métrica e o limiar para considerar um resultado “bom” variam conforme a aplicação. Em aplicações médicas, por exemplo, onde a precisão

é crítica, muitas vezes exige-se um IoU superior a 0,9 para considerar a segmentação aceitável. Já em aplicações de visão para robótica, onde o sistema pode corrigir erros em etapas posteriores, limiares mais baixos podem ser tolerados.

Para visualizar a influência de diferentes métodos e parâmetros na segmentação, é útil comparar uma mesma imagem processada por técnicas distintas. Imagine uma fotografia aérea de uma plantação: ao aplicar uma segmentação por limiar fixo, obtém-se uma separação simples entre áreas verdes e não verdes. Já com um método de clustering não supervisionado, pode-se observar que diferentes tipos de vegetação são identificados separadamente. Por outro lado, a aplicação de um método baseado em bordas pode destacar apenas as linhas de plantio, omitindo áreas internas homogêneas.



Figura 2 - Diferentes métodos de segmentação aplicados à mesma imagem
Fonte: Builtin (2025)

Os fundamentos abordados aqui se baseiam em obras clássicas como: *Digital Image Processing* de Gonzalez e Woods (2018), que detalham a matemática por trás da conectividade e das medidas de similaridade; e *Computer Vision: Algorithms and Applications* de Szeliski (2022), que discute as implicações práticas dessas escolhas no desempenho de sistemas reais. Essas referências deixam claro que dominar os conceitos básicos de pixels, conectividade, similaridade e métricas de avaliação é indispensável para que possamos compreender, implementar e melhorar algoritmos de segmentação em qualquer contexto.

Ao avançarmos para as próximas seções, veremos como esses fundamentos se concretizam em técnicas específicas como limiarização, métodos baseados em regiões e abordagens baseadas em bordas. No entanto, é importante manter em mente que a compreensão desses conceitos estruturais é o que permitirá ajustar parâmetros, escolher o método adequado e interpretar resultados de forma crítica e fundamentada.

FIAP

3 MÉTODOS DE LIMIARIZAÇÃO (THRESHOLDING)

A limiarização é, provavelmente, a forma mais simples e direta de segmentação de imagens, sendo amplamente utilizada como uma primeira abordagem para separar regiões de interesse do fundo. A ideia central é muito intuitiva: escolher um valor de intensidade como limite (*threshold*) e classificar cada pixel como pertencente ao objeto ou ao fundo, dependendo se seu valor está acima ou abaixo desse limite.

Se considerarmos uma imagem em tons de cinza, o procedimento funciona assim: pixels mais claros (com valores mais altos) podem representar o objeto e serão marcados como “1” (branco), enquanto pixels mais escuros (com valores mais baixos) podem representar o fundo e serão marcados como “0” (preto). O resultado é uma imagem binária, na qual a segmentação se torna clara e fácil de processar em etapas posteriores, como extração de contornos ou cálculo de características.

3.1 Limiarização global

Na limiarização global (*global thresholding*), escolhemos um único valor de limiar para toda a imagem. Esse valor pode ser definido manualmente, com base na análise visual do histograma de intensidades, ou determinado automaticamente por um algoritmo.

Por exemplo, imagine uma imagem de moedas sobre um fundo preto e bem iluminado. Se o histograma mostrar dois picos distintos — um para os pixels do fundo e outro para os pixels das moedas —, podemos escolher um valor intermediário entre os picos como limiar. Todos os pixels com valor acima desse limiar serão classificados como moedas, e todos abaixo, como fundo.

Matematicamente, o processo é descrito como:

$$g(x,y) = \begin{cases} 1, se\ f(x,y) \geq T \\ 0, se\ f(x,y) < T \end{cases}$$

Em que: $f(x,y)$ é o valor de intensidade do pixel na posição (x,y); T é o limiar escolhido; e $g(x,y)$ é o pixel resultante na imagem binária.

A principal limitação da limiarização global é que ela assume iluminação uniforme e contraste consistente em toda a imagem. Em imagens com iluminação desigual, sombras ou gradientes de brilho, um único valor de limiar dificilmente separa de forma adequada o objeto do fundo.

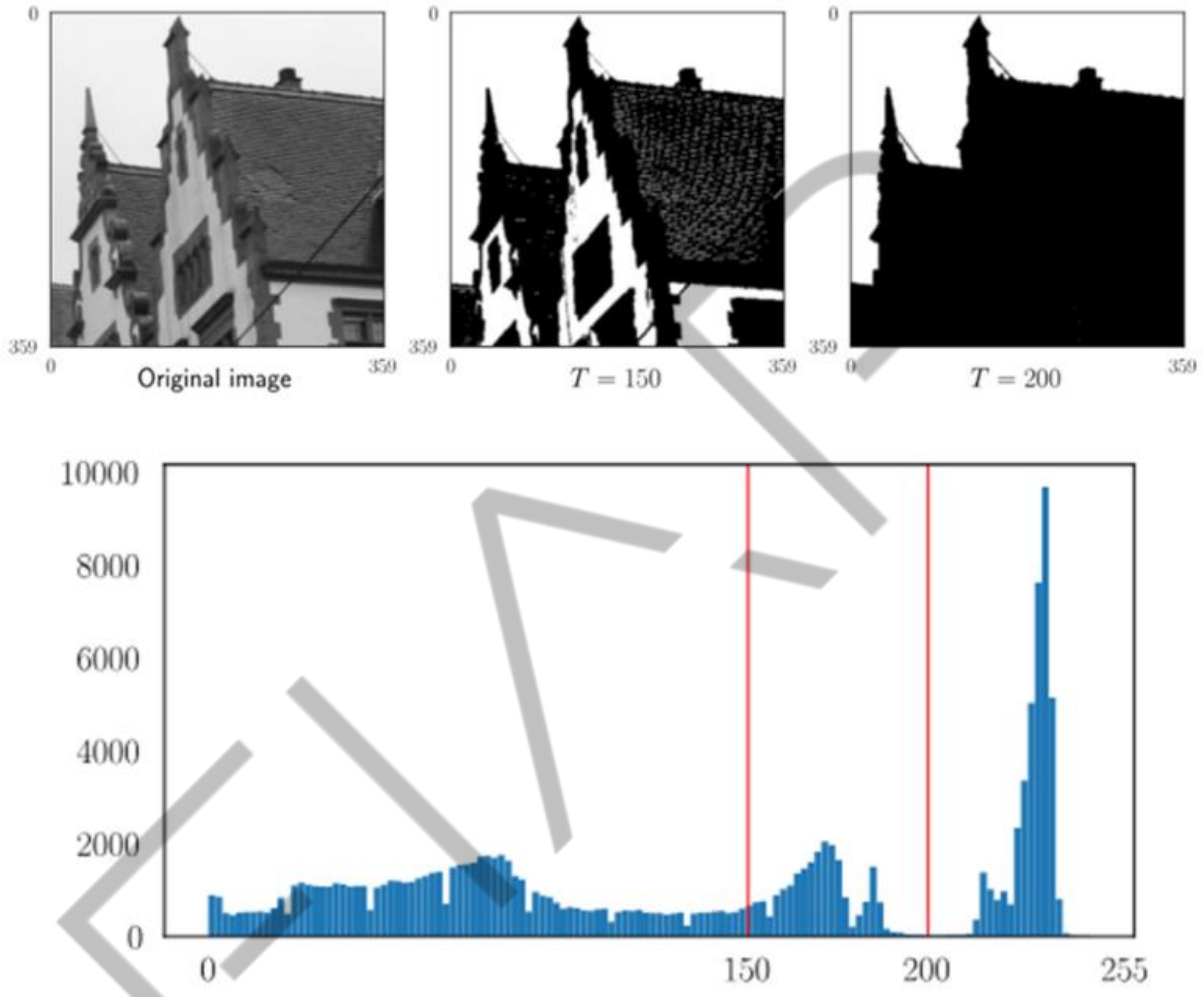


Figura 3 - Limiarização com diferentes limiares T
Fonte: GitHub (2023), adaptado por FIAP (2025)

3.2 Método de Otsu

O método de Otsu, proposto por Nobuyuki Otsu em 1979, é uma técnica de limiarização global automática que busca maximizar a separação entre duas classes de pixels, objeto e fundo, sem a necessidade de intervenção humana. A ideia é escolher o limiar que maximize a variância entre classes (between-class variance) ou, equivalentemente, minimize a variância dentro das classes (within-class variance).

Segue o passo a passo do funcionamento:

- **Passo 1:** calcula-se o histograma da imagem e a probabilidade de ocorrência de cada nível de intensidade.
- **Passo 2:** para cada possível valor de limiar t, divide-se a imagem em duas classes: C_0 (pixels com intensidade ≤ t) e C_1 (pixels com intensidade > t).
- **Passo 3:** calcula-se a média e a variância de cada classe, bem como o peso (proporção de pixels) de cada uma.
- **Passo 4:** a variância entre classes é dada por: $\sigma_b^2(t) = \omega_0(t) \cdot \omega_1(t) \cdot [\mu_0(t) - \mu_1(t)]^2$, onde $\omega_0(t)$ e $\omega_1(t)$ são as proporções de pixels nas classes 0 e 1, e $\mu_0(t)$ e $\mu_1(t)$ são as médias de intensidade de cada classe.
- **Passo 5:** o valor de t que maximiza $\sigma_b^2(t)$ é escolhido como limiar ótimo.

O método de Otsu funciona muito bem em imagens com histograma bimodal (dois picos bem definidos), mas pode falhar quando há múltiplos objetos com intensidades variadas ou iluminação irregular.

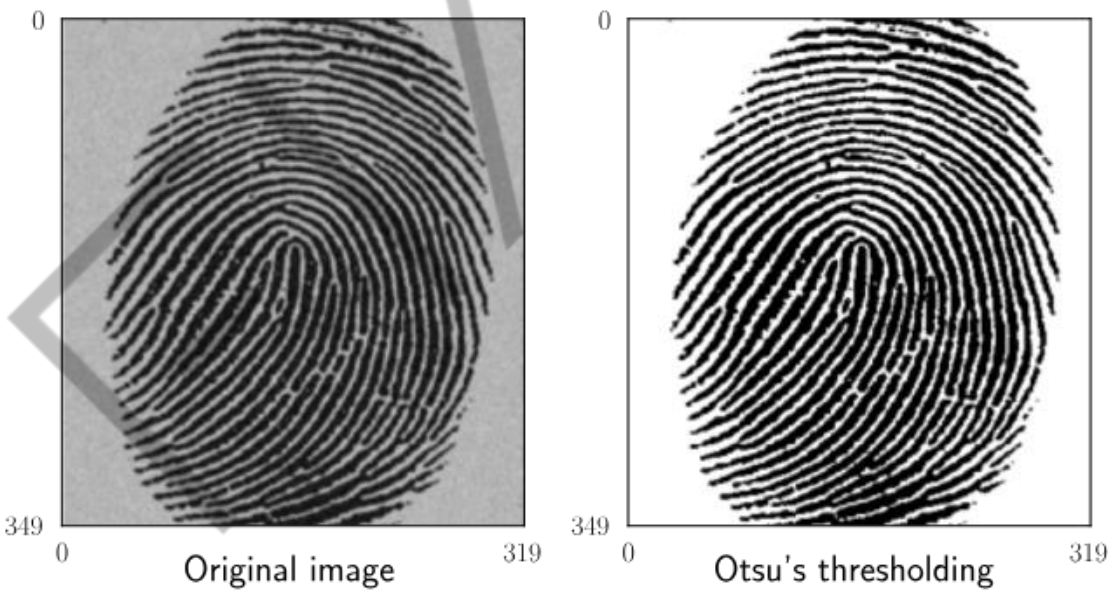


Figura 4 - Limiarização com Otsu
Fonte: GitHub (2023)

3.3 Limiarização adaptativa

Quando a iluminação varia significativamente em diferentes partes da imagem, a limiarização adaptativa (*adaptive thresholding*) é mais apropriada. Em vez de usar

um único valor global, ela calcula um limiar diferente para cada região da imagem, considerando a vizinhança de cada pixel.

Existem dois métodos principais:

- **Média adaptativa (mean adaptive):** o limiar para cada pixel é a média das intensidades de seus vizinhos, subtraído de um valor constante C para compensar variações.
- **Gaussiana adaptativa (Gaussian adaptive):** semelhante à média adaptativa, mas os vizinhos são ponderados por uma função Gaussiana, dando mais importância aos pixels mais próximos.

Essas abordagens são especialmente úteis em aplicações como reconhecimento de texto manuscrito (OCR) em documentos antigos, onde a iluminação e o contraste podem variar bastante entre as áreas da página.

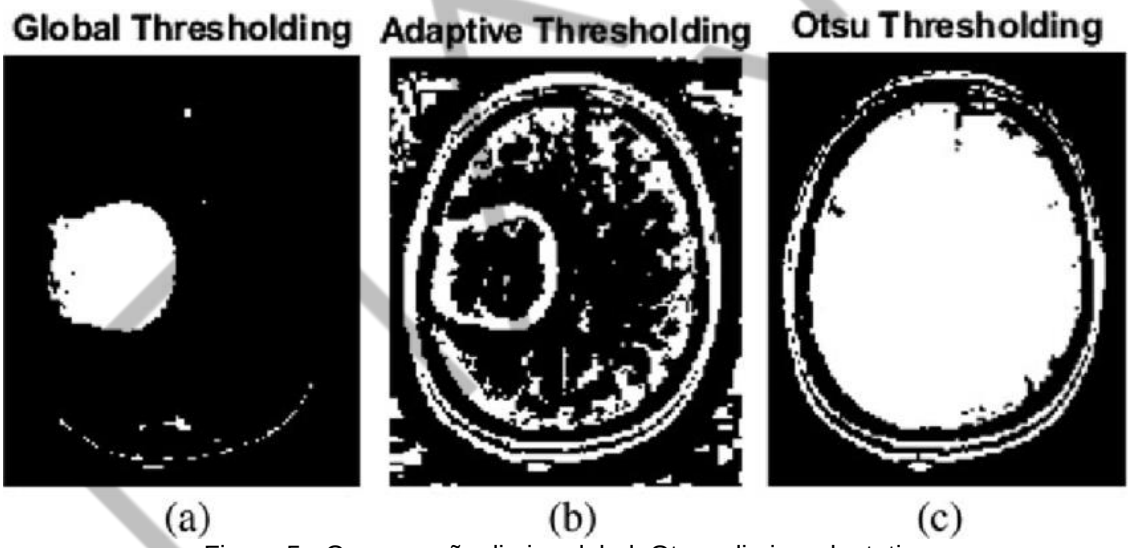


Figura 5 - Comparação: limiar global, Otsu e limiar adaptativo
Fonte: ResearchGate (2021)

3.4 Pontos fortes e fracos

A limiarização é rápida, simples e computacionalmente leve, sendo adequada para aplicações em tempo real ou para pré-processamento antes de métodos mais sofisticados. No entanto, ela depende fortemente da qualidade da iluminação e do contraste da imagem. Os métodos globais funcionam bem em imagens uniformes, mas podem falhar em condições reais, enquanto o método adaptativo — embora mais robusto — é mais sensível a ruídos e demanda mais processamento.

3.5 Exemplos de Código com OpenCV

A biblioteca OpenCV oferece algumas funções prontas para aplicar todos esses métodos.

```
import cv2
import numpy as np

# Carrega imagem em tons de cinza
img = cv2.imread('documento.png', cv2.IMREAD_GRAYSCALE)

# Limiarização global
_, thresh_global = cv2.threshold(img, 127, 255,
cv2.THRESH_BINARY)

# Método de Otsu
_, thresh_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

# Limiarização adaptativa (média)
thresh_adapt_mean = cv2.adaptiveThreshold(img, 255,
cv2.ADAPTIVE_THRESH_MEAN_C,
                                          cv2.THRESH_BINARY, 11,
2)

# Limiarização adaptativa (Gaussiana)
thresh_adapt_gauss = cv2.adaptiveThreshold(img, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                           cv2.THRESH_BINARY,
11, 2)

# Exibição
cv2.imshow("Global", thresh_global)
cv2.imshow("Otsu", thresh_otsu)
cv2.imshow("Adaptive Mean", thresh_adapt_mean)
cv2.imshow("Adaptive Gaussian", thresh_adapt_gauss)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Código-fonte 1 - Filtros de Limiarização
Fonte: Elaborado pelo autor (2025)

O dataset é baixado automaticamente e carregado com normalização dos pixels para o intervalo [-1, 1], o que facilita o treinamento dos modelos.

4 SEGMENTAÇÃO BASEADA EM REGIÕES

A segmentação baseada em regiões parte de um princípio diferente da limiarização: em vez de determinar limites de intensidade que separam o objeto do fundo, ela agrupa pixels que apresentam similaridade segundo algum critério, formando regiões coerentes que correspondem a partes significativas da cena. Essa abordagem é particularmente útil quando o objeto e o fundo possuem sobreposição nos valores de intensidade ou quando queremos manter a coerência espacial, evitando que pixels isolados sejam mal classificados.

O conceito central é que uma região é um conjunto de pixels que compartilham propriedades semelhantes, seja intensidade, cor, textura ou até mesmo características mais complexas aprendidas por redes neurais. A segmentação por regiões busca identificar e agrupar essas áreas homogêneas, preservando a estrutura da imagem.

Entre os métodos mais conhecidos dessa categoria estão o crescimento de regiões (*region growing*), a técnica de divisão e fusão (*region splitting and merging*) e a segmentação por mean-shift.

4.1 Crescimento de regiões (Region growing)

O crescimento de regiões é um dos métodos mais intuitivos. Ele começa com a seleção de pontos sementes (*seed points*), que podem ser escolhidos manualmente ou detectados automaticamente. A partir dessas sementes, o algoritmo analisa os pixels vizinhos e decide se eles devem ser incluídos na região com base em um critério de similaridade, normalmente, uma diferença máxima permitida de intensidade ou cor.

O processo segue um ciclo iterativo: a cada passo, adicionam-se novos pixels vizinhos que satisfazem o critério, e o conjunto da região cresce até que nenhum novo pixel possa ser incorporado. Esse método é especialmente útil em imagens onde as regiões de interesse possuem transições suaves e homogêneas, como em imagens médicas para segmentar órgãos ou tumores.

Um exemplo simples é segmentar uma fruta clara (como uma laranja iluminada pelo sol) em uma imagem com fundo verde de folhas. Se selecionarmos um pixel no

centro da fruta como semente e definirmos uma tolerância de cor apropriada, o crescimento de regiões irá expandir-se até cobrir toda a área da laranja, mas não entrará nas folhas, que têm cor e textura diferentes.

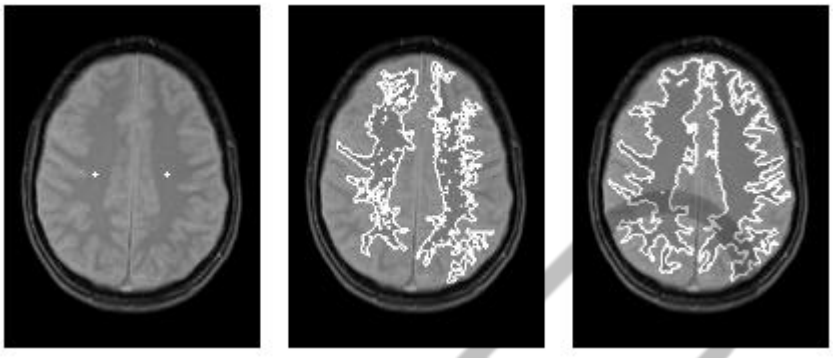


Figura 6 - *Region Growing* aplicada a exame de imagem do cérebro humano
Fonte: ResearchGate (2024)

A principal limitação desse método é a sensibilidade à escolha das sementes e ao critério de similaridade. Um limiar muito baixo pode gerar regiões incompletas, enquanto um limiar muito alto pode englobar partes indesejadas.

4.2 Divisão e fusão (region splitting and merging)

Enquanto o crescimento de regiões começa pequeno e depois se expande, a técnica de divisão e fusão segue o caminho oposto e complementar. Primeiramente, o processo trata toda a imagem como uma única região. Em seguida, verifica-se se essa região é homogênea segundo um critério pré-definido. Se não for, ela é dividida em sub-regiões (por exemplo, em quatro partes iguais).

Cada sub-região é então analisada novamente, e o processo de divisão continua até que todas as regiões atendam ao critério de homogeneidade. Após essa etapa, o algoritmo realiza a fusão de regiões vizinhas que sejam semelhantes o suficiente, garantindo que regiões fragmentadas desnecessariamente sejam reunidas.

Esse método é útil quando não sabemos de antemão onde estão as sementes ou quando a cena apresenta variações complexas, mas ainda possui áreas homogêneas internamente. No entanto, pode ser mais custoso computacionalmente, já que envolve múltiplas verificações e fusões.

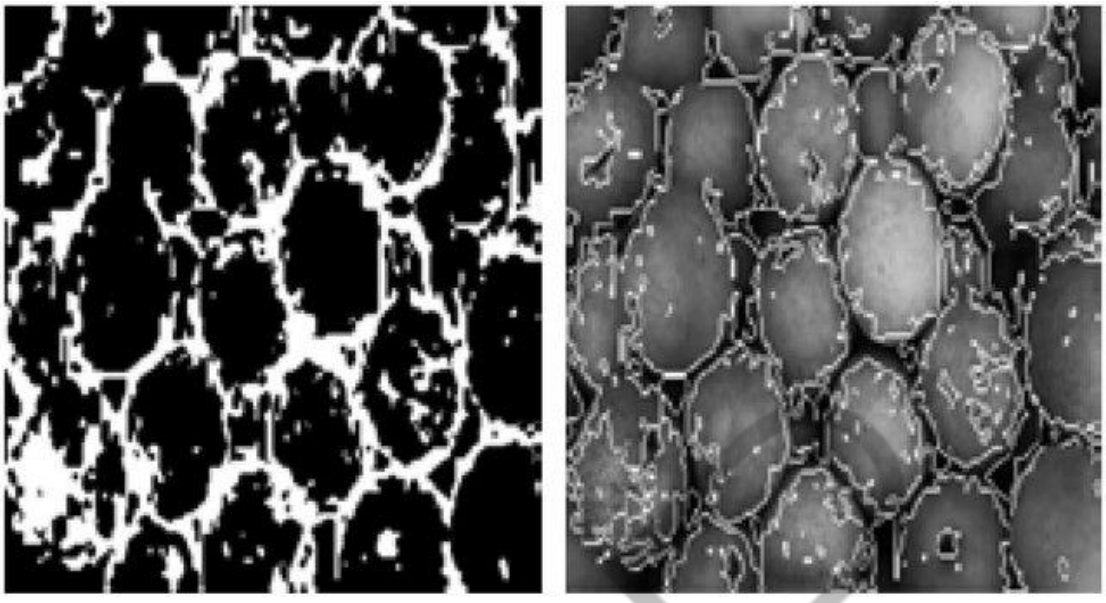


Figura 7 - Segmentação *Split and Merge* aplicado ao diagnóstico de melanoma
Fonte: ResearchGate (2014)

4.3 Segmentação por Mean-Shift

O algoritmo mean-shift é um método não paramétrico que procura modos (picos) de densidade no espaço das características da imagem. Em segmentação, considera-se um espaço que combina a posição espacial dos pixels e seus valores de cor.

O processo inicia selecionando uma janela de busca em torno de cada pixel e calculando o ponto médio ponderado das cores (ou intensidades) dos pixels dentro dessa janela. Esse ponto médio desloca-se iterativamente na direção da maior densidade de pixels semelhantes, até convergir para um ponto estável.

Atenção: pixels que convergem para o mesmo modo são agrupados na mesma região.

O mean-shift é particularmente bom para segmentar imagens com objetos bem definidos em termos de cor, como frutas coloridas, roupas ou placas de trânsito.

No OpenCV, a função cv2.pyrMeanShiftFiltering aplica uma versão eficiente desse método, usando pirâmides de imagem para reduzir a complexidade computacional.

Exemplo de código:

```python
import cv2
from google.colab.patches import cv2_imshow

# Carrega imagem
img = cv2.imread('cerebro.jpg')

# Aplica filtragem mean-shift
shifted = cv2.pyrMeanShiftFiltering(img, sp=50, sr=75)

# Exibe resultado
cv2_imshow(img)
cv2_imshow(shifted)
```

Código-fonte 2 - Mean shift no opencv
Fonte: Elaborado pelo autor (2025)

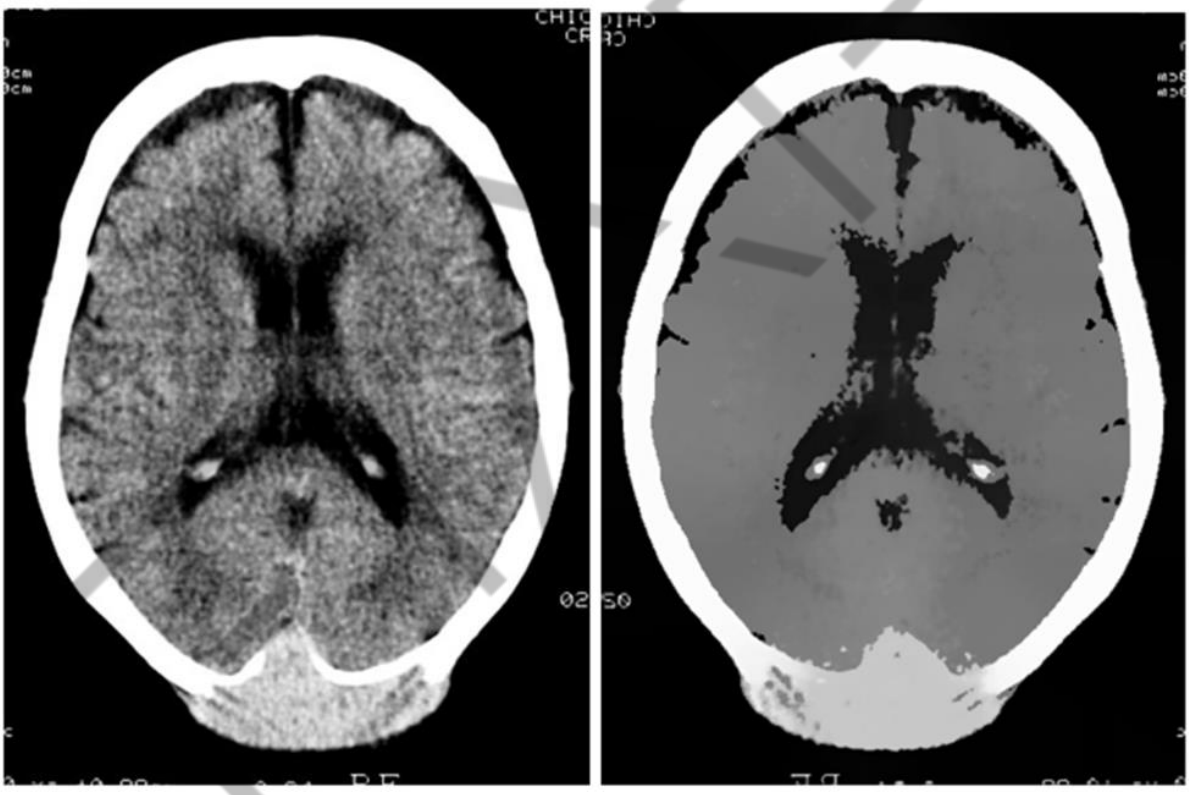


Figura 8 - Segmentação Mean-shift aplicada a tomografia cerebral. Original (esquerda) e segmentada (direita)
Fonte: Unicamp (2025), adaptado pelo autor (2025)

Neste exemplo, o parâmetro *sp* define o raio espacial da janela e *sr* o raio no espaço de cores. Valores maiores resultam em regiões mais suaves e agrupadas, enquanto valores menores preservam mais detalhes.

5 SEGMENTAÇÃO BASEADA EM BORDAS (EDGE-BASED SEGMENTATION)

A segmentação baseada em bordas parte de um princípio fundamental: a maioria dos objetos no mundo real apresenta descontinuidades abruptas em intensidade, cor ou textura na transição entre sua superfície e o fundo. Essas descontinuidades aparecem na imagem como bordas, que podem ser detectadas analisando-se o gradiente local da intensidade.

A ideia é relativamente direta: se um pixel apresenta uma variação acentuada em relação aos seus vizinhos, ele provavelmente pertence a uma borda. Identificar e realçar essas bordas permite isolar objetos, contornar suas formas e, em muitos casos, preparar o terreno para etapas posteriores de segmentação ou reconhecimento.

Esse tipo de abordagem é amplamente utilizado em cenários como: inspeção industrial (para verificar contornos de peças), reconhecimento de caracteres (onde o formato das letras é essencial) e até na medicina (para delinear vasos sanguíneos ou órgãos).

5.1 Operadores de gradiente

O cálculo do gradiente em uma imagem mede a variação da intensidade luminosa ao longo das direções horizontal e vertical. Para isso, utilizam-se operadores de convolução que realçam essas variações.

Entre os mais conhecidos estão os operadores **Sobel**, **Prewitt** e **Scharr**. Todos eles funcionam aplicando pequenos filtros (máscaras) que calculam uma aproximação da derivada da imagem.

O operador Sobel, por exemplo, utiliza duas máscaras diferentes para calcular as variações horizontais e verticais. O resultado final é obtido combinando esses gradientes para encontrar a magnitude e a direção da borda.

O Prewitt é semelhante, mas utiliza coeficientes mais simples — o que o torna levemente mais rápido, porém menos robusto ao ruído. Já o Scharr é uma versão

otimizada para obter melhor precisão em gradientes suaves, sendo particularmente útil quando a detecção de bordas finas é crítica.

Exemplo simples de aplicação do Sobel no OpenCV:

```
import cv2
import numpy as np

img = cv2.imread('cerebro.jpg', cv2.IMREAD_GRAYSCALE)

sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)  # Gradiente X
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)  # Gradiente Y

magnitude = np.sqrt(sobelx**2 + sobely**2)
magnitude = cv2.convertScaleAbs(magnitude)

cv2_imshow(img)
cv2_imshow(magnitude)
```

Código-fonte 3 - Aplicação do Sobel em tomografia do cérebro
Fonte: Elaborado pelo autor (2025)

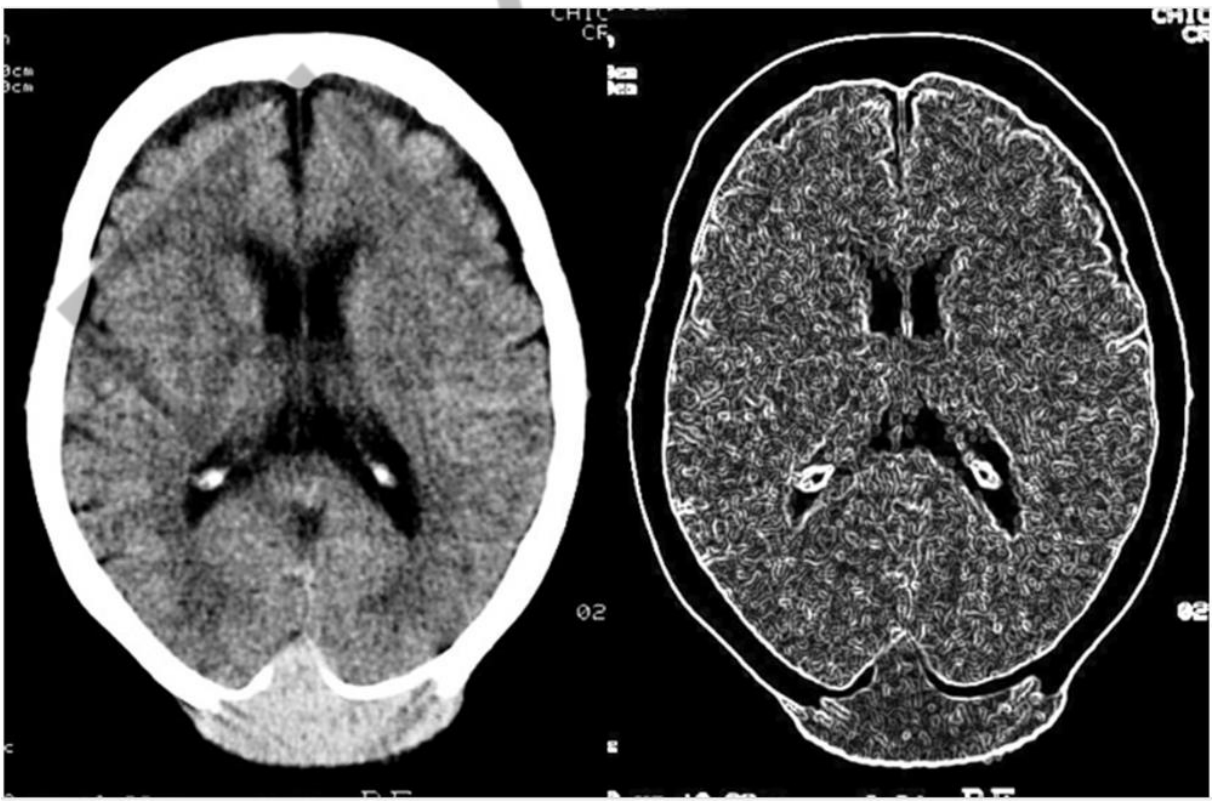


Figura 9 - Sobel aplicada a tomografia cerebral. Original (esquerda) e Sobel (direita)
Fonte: Unicamp (2025), adaptado pelo autor (2025)

5.2 Detector de Bordas de Canny

O algoritmo de Canny é considerado um dos detectores de bordas mais robustos, sendo amplamente utilizado devido à sua capacidade de detectar bordas finas e contínuas mesmo em imagens com ruído. Ele segue um pipeline bem definido:

- **Suavização Gaussiana:** antes de calcular o gradiente, a imagem é suavizada com um filtro Gaussiano para reduzir ruído. Isso é essencial, pois ruídos podem gerar bordas falsas.
- **Cálculo do Gradiente:** o gradiente é calculado, normalmente usando Sobel, para encontrar magnitude e direção das bordas.
- **Supressão Não Máxima (Non-Maximum Suppression):** mantém apenas os pixels que representam o pico real do gradiente na direção da borda, afinando as linhas detectadas.
- **Histerese de Limiar (Hysteresis Thresholding):** aplica dois limiares: um alto e um baixo. Bordas fortes (acima do limiar alto) são mantidas; bordas fracas (entre os dois valores) são mantidas apenas se estiverem conectadas a bordas fortes; e bordas muito fracas são descartadas.

Essa estratégia permite manter a continuidade das bordas e evitar falsas detecções.

Exemplo de aplicação no OpenCV:

```
import cv2

img = cv2.imread('cerebro.jpg', cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(img, threshold1=100, threshold2=200)

cv2_imshow(img)
cv2_imshow(edges)
```

Código-fonte 4 - Aplicação do Canny em tomografia do cérebro
Fonte: Elaborado pelo autor (2025)

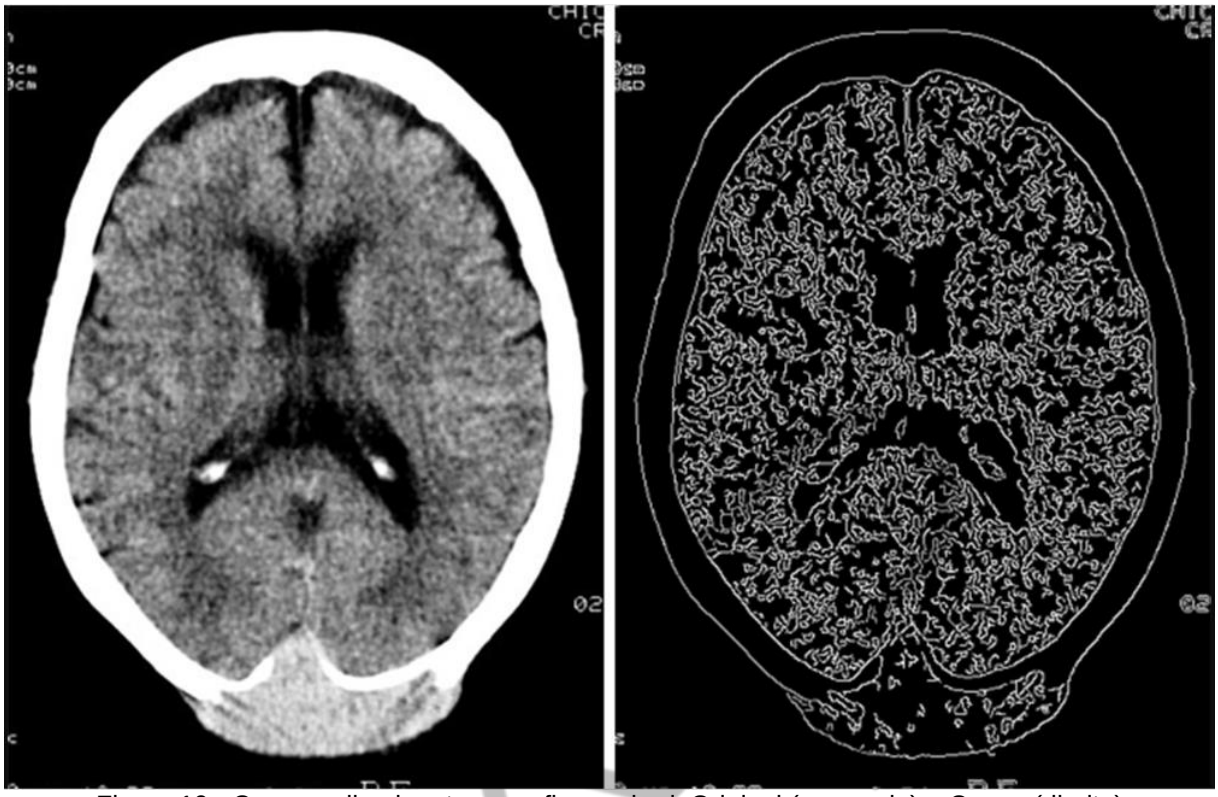


Figura 10 - Canny aplicado a tomografia cerebral. Original (esquerda) e Canny (direita)
Fonte: Unicamp (2025), adaptado pelo autor (2025)

5.3 Detecção de Contornos

Uma vez detectadas as bordas, é possível usar funções de rastreamento de contorno para extrair os limites completos dos objetos. No OpenCV, isso é feito com a função cv2.findContours, que retorna uma lista de contornos, cada um representado por uma sequência de pontos.

Além de detectar o contorno, o OpenCV fornece uma hierarquia que indica relações entre eles, por exemplo, se um contorno está dentro de outro — útil para detectar furos ou objetos internos.

Exemplo prático no OpenCV:

```
      import cv2

      img = cv2.imread('cerebro.jpg')
      gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
      edges = cv2.Canny(gray, 150, 450)

      contours,     hierarchy     =     cv2.findContours(edges,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

      cv2_imshow(img)
      print(f"Contornos detectadas: {len(contours)}")
      cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
```

Código-fonte 5 - Aplicação do Canny + detecção de contornos em tomografia do cérebro
Fonte: Elaborado pelo autor (2025)

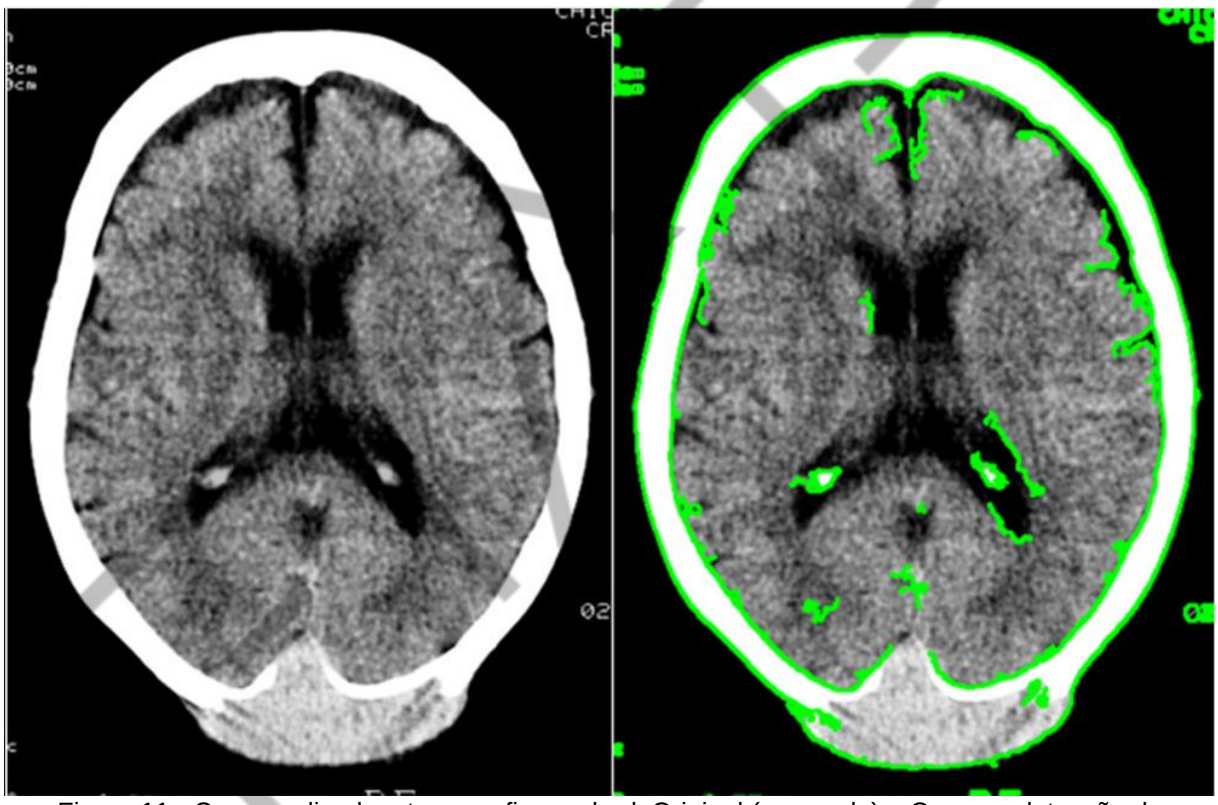


Figura 11 - Canny aplicado a tomografia cerebral. Original (esquerda) e Canny + detecção de contornos (direita)
Fonte: Unicamp (2025), adaptado pelo autor (2025)

6 FUNDAMENTOS DA EXTRAÇÃO DE CARACTERÍSTICAS (FEATURE EXTRACTION BASICS)

Quando um sistema de Visão Computacional recebe uma imagem, o que ele enxerga inicialmente é apenas uma matriz de números representando intensidades de pixels (em tons de cinza) ou valores RGB (em imagens coloridas). Trabalhar diretamente com esses valores crus geralmente não é a abordagem mais eficiente para tarefas de reconhecimento ou classificação, pois eles não capturam, de forma compacta e robusta, as informações mais relevantes sobre os objetos presentes na cena. É aqui que entra o conceito de extração de características.

A extração de características consiste em transformar uma imagem em um conjunto de descritores ou atributos que representem, de forma condensada e informativa, o conteúdo visual que nos interessa. Essas características funcionam como um “resumo inteligente” da imagem, preservando as informações mais importantes para a tarefa em questão e descartando redundâncias ou detalhes irrelevantes.

6.1 Características locais versus características globais

As características podem ser divididas em duas categorias principais: locais e globais.

As características locais descrevem regiões específicas da imagem, geralmente ao redor de pontos de interesse ou “*keypoints*”. Elas são muito úteis quando precisamos identificar ou comparar objetos mesmo que estejam em diferentes posições, orientações ou escalas dentro da imagem. Por exemplo, se quisermos reconhecer um monumento em fotos tiradas de ângulos diferentes, é mais robusto extrair características locais (como as usadas no SIFT ou ORB) que permitam fazer correspondências ponto a ponto entre as imagens.

Já as características globais descrevem a imagem como um todo, fornecendo uma visão mais geral de suas propriedades. Exemplos incluem histogramas de cor, descritores de textura e momentos estatísticos da imagem. Essas características são úteis quando a informação relevante está distribuída de forma mais uniforme ou

quando a localização exata dos padrões não é crucial. Por exemplo, para diferenciar imagens de praias e florestas, um histograma de cores globais pode ser suficiente.

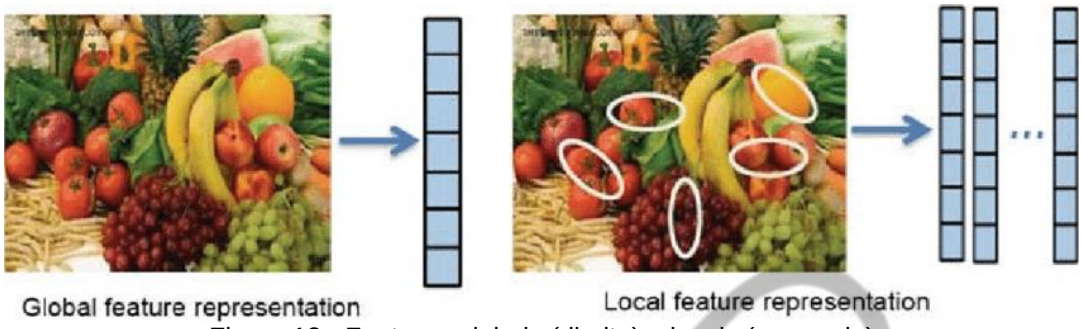


Figura 12 - Features globais (direita) e locais (esquerda)
Fonte: Medium (2023)

6.2 Propriedades desejáveis de características

Para serem eficazes, as características extraídas devem possuir certas propriedades fundamentais. A primeira delas é a **invariância à escala**, ou seja, a capacidade de descrever o mesmo objeto independentemente de ele aparecer maior ou menor na imagem.

Essa propriedade é essencial, por exemplo, para reconhecer um carro visto de perto ou de longe.

Outra propriedade desejável é a **invariância à rotação**. Muitos objetos podem aparecer girados em diferentes imagens, e um bom descritor precisa reconhecer que se trata do mesmo objeto.

Essa é uma propriedade fundamental, por exemplo, na inspeção de peças industriais em uma esteira, onde a orientação pode variar, mas o sistema deve detectá-las corretamente.

A **invariância à iluminação** também é crucial, pois mudanças na intensidade de luz ou na presença de sombras não devem impedir que a característica represente corretamente o objeto.

Isso é particularmente importante em aplicações externas, como reconhecimento de placas de veículos, onde as condições de iluminação variam ao longo do dia.

Por fim, além da invariância, as características devem ser distintivas — ou seja, devem capturar detalhes que as tornem únicas e facilmente diferenciáveis de outras características na imagem.

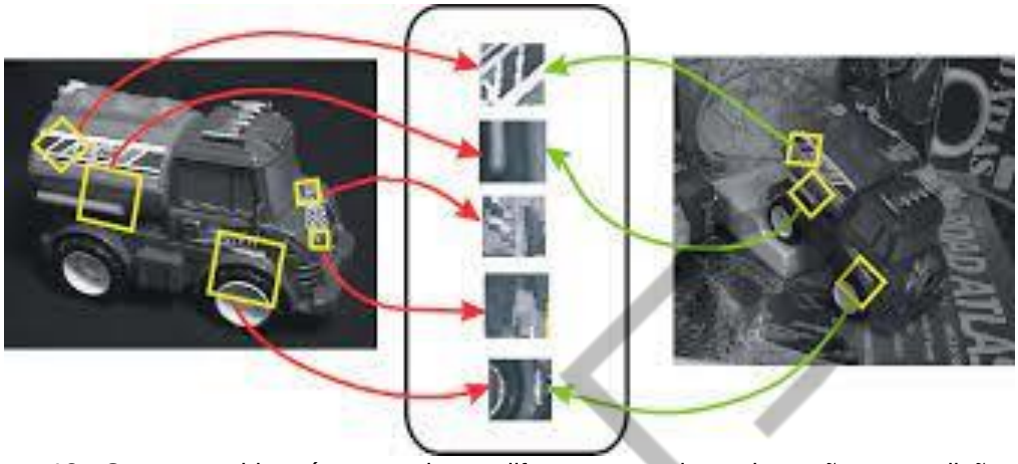


Figura 13 - O mesmo objeto é capturado em diferentes escalas, orientações e condições de iluminação, com keypoints detectados de forma consistente
Fonte: Medium (2019)

6.3 Descritores *versus* valores brutos de pixels

Enquanto uma imagem bruta pode ser representada por milhões de valores de pixels, um descritor de características é tipicamente um vetor de dimensões muito menores, contendo informações específicas e discriminativas.

Por exemplo, em um detector de bordas como o Canny, poderíamos considerar como característica o número total de pixels de borda detectados. No entanto, em um método mais avançado como o SIFT, cada ponto-chave (*keypoint*) é descrito por um vetor de 128 valores que codificam a distribuição de gradientes na vizinhança desse ponto.

A vantagem dos descritores é que eles permitem comparar regiões ou objetos de diferentes imagens de forma muito mais eficiente e robusta do que comparar todos os pixels diretamente. Essa redução na dimensionalidade também é fundamental para acelerar algoritmos de busca e classificação.

6.4 Fluxo no pipeline de Visão Computacional

Na maioria dos sistemas modernos de Visão Computacional, a extração de características aparece após a etapa de pré-processamento e segmentação. Primeiro, a imagem é ajustada para remover ruído e normalizar cores. Em seguida, a segmentação separa o objeto de interesse do fundo. Por fim, as características são extraídas para representar esse objeto de forma única e compacta.

Um exemplo prático pode ser visto no diagnóstico por imagem em medicina. Em uma ressonância magnética cerebral, primeiro isolamos a região do tumor usando segmentação, depois extraímos características como textura, forma e intensidade média, que servirão como entrada para um classificador capaz de diferenciar tumores benignos de malignos.

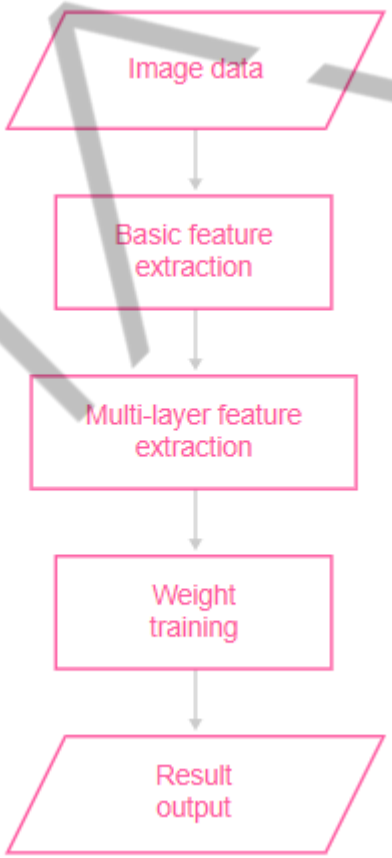


Figura 14 - Pipeline genérico de um sistema de Visão Computacional
Fonte: ResearchGate (2022), adaptado por FIAP (2025)

7 DESCRITORES LOCAIS DE CARACTERÍSTICAS (LOCAL FEATURE DESCRIPTORS)

A extração de características locais é um dos pilares da Visão Computacional clássica, especialmente antes da popularização das redes neurais profundas. Esses descritores têm como objetivo encontrar pontos-chave (*keypoints*) em uma imagem que sejam únicos e fáceis de identificar, independentemente de transformações como mudança de escala, rotação, iluminação ou perspectiva.

Métodos como **SIFT**, **SURF** e **ORB** tornaram-se referências nesse campo, cada um com suas particularidades, vantagens e desvantagens. Nesta seção, exploraremos em detalhes o funcionamento desses três algoritmos, sua teoria, aplicações práticas e implementações com OpenCV.

7.1 SIFT (Scale-Invariant Feature Transform)

O SIFT, proposto por David Lowe em 1999 e aprimorado em 2004, é um dos descritores de características mais conhecidas e robustas da Visão Computacional. Sua principal força está na invariância à escala e à rotação, permitindo que um mesmo objeto seja reconhecido mesmo quando aparece em diferentes tamanhos ou orientações na imagem.

O processo do SIFT pode ser dividido em quatro etapas principais:

- **Etapa 1 – Construção do espaço de escalas (Scale-Space Theory):** o SIFT utiliza a teoria do espaço de escalas para detectar pontos-chave que sejam estáveis em diferentes níveis de zoom. Para isso, cria-se uma pirâmide de imagens borradas com diferentes valores de sigma (σ) em um filtro Gaussiano.
- **Etapa 2 – Diferença de Gaussianas (DoG – Difference of Gaussians):** para encontrar regiões de interesse, calcula-se a diferença entre imagens borradas consecutivas da pirâmide. Essa operação realça estruturas que se mantêm estáveis entre diferentes escalas.

- **Etapa 3 – Localização precisa dos pontos-chave:** cada máximo ou mínimo local detectado no espaço de escalas é analisado para verificar se é um ponto estável. Picos fracos ou pontos causados por bordas pouco relevantes são descartados.
- **Etapa 4 – Criação do descritor:** ao redor de cada ponto-chave, calcula-se um histograma de gradientes orientados. Esses valores são normalizados para garantir robustez contra variações de iluminação. O resultado é um vetor de 128 dimensões que descreve o padrão local.

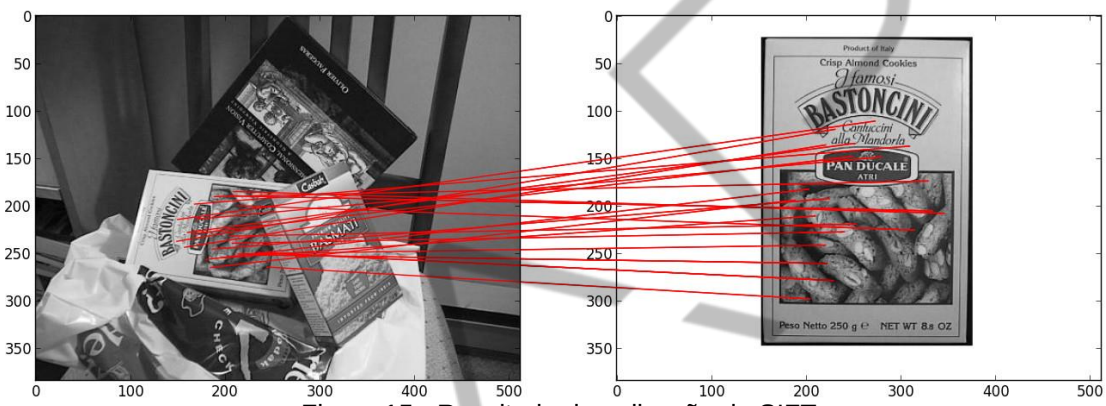


Figura 15 - Resultado da aplicação do SIFT
Fonte: Python Hosted (2025)

O SIFT é extremamente robusto, mas sua complexidade computacional é relativamente alta, tornando-o mais adequado para aplicações offline ou em tempo não crítico, como análise forense de imagens ou reconhecimento de patrimônios históricos.

7.2 SURF (Speeded Up Robust Features)

O SURF, introduzido em 2006 por Herbert Bay e colaboradores, foi desenvolvido como uma alternativa mais rápida ao SIFT, mantendo um nível semelhante de robustez.

A principal diferença está na substituição das convoluções Gaussianas do SIFT por aproximações baseadas em filtros de caixa (*box filters*), que podem ser calculados de forma extremamente rápida usando imagens integrais.

Além disso, o SURF utiliza determinantes da matriz Hessiana para detectar pontos de interesse, o que também acelera o processo de detecção. Seu descritor é

menor e mais rápido de calcular, mas ainda mantém boa capacidade de invariância à escala e rotação.

Apesar de sua eficiência, o SURF também é protegido por patente, o que restringe seu uso em aplicações comerciais sem licenciamento, assim como o SIFT (até 2020, quando sua patente expirou).

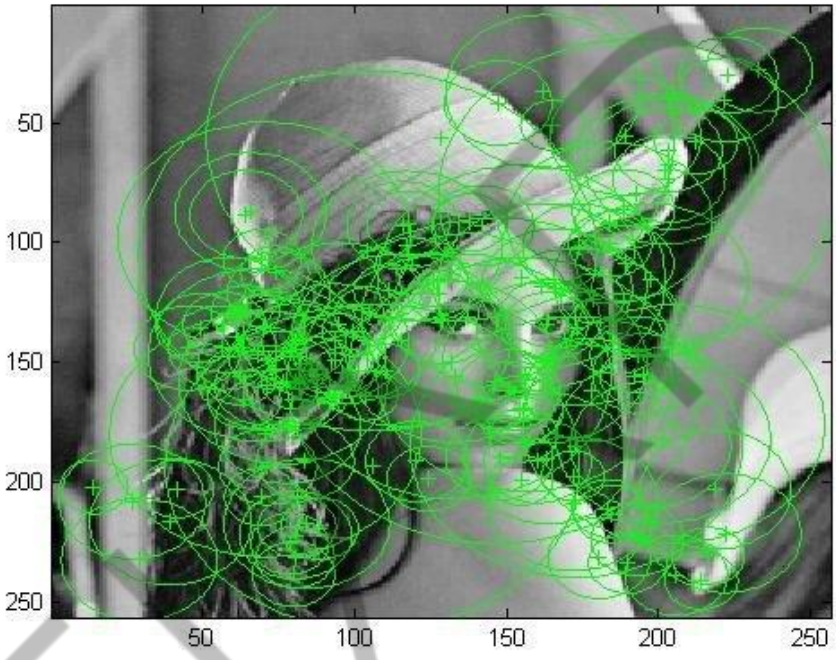


Figura 16 - Resultado da aplicação do SURF
Fonte: ResearchGate (2014)

7.3 ORB (Oriented FAST and Rotated BRIEF)

O ORB, desenvolvido pelo laboratório de pesquisa da OpenCV em 2011, combina dois métodos: **FAST** (para detecção rápida de pontos de interesse) e **BRIEF** (para descrição de características).

Sua principal vantagem é que ele é livre de patentes e extremamente rápido, tornando-se ideal para aplicações em tempo real, como rastreamento de objetos em sistemas embarcados, realidade aumentada e robótica móvel.

Diferentemente do SIFT e SURF, que produzem descritores vetoriais de valores reais, o ORB gera descritores binários, permitindo comparações muito rápidas usando métricas como *Hamming distance*.

Embora o ORB seja menos robusto a grandes mudanças de iluminação ou transformações geométricas muito severas, seu custo computacional reduzido o torna amplamente utilizado em aplicações práticas onde o tempo de resposta é crítico.

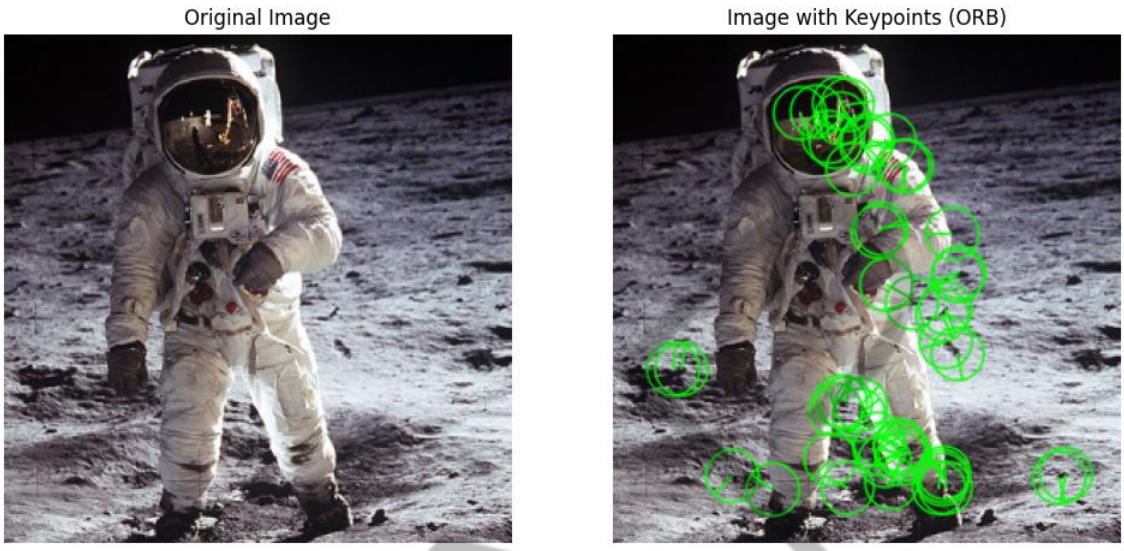


Figura 17 - Resultado da aplicação do ORB
Fonte: Roboflow (2024)

7.4 Comparação entre SIFT, SURF e ORB

O quadro resume as principais diferenças:

Algoritmo	Invariância	Velocidade	Tipo descritor	Licenciamento
SIFT	Escala, rotação, iluminação	Lento	Vetorial (128 float)	Patente expirada
SURF	Escala, rotação, iluminação	Médio	Vetorial (64 float)	Patente ativa
ORB	Escala, rotação	Rápido	Binário (256 bits)	Open Source

Quadro 1 - Principais diferenças entre SIFT, SURF e ORB
Fonte: Elaborado pelo autor (2025)

7.5 Implementação em OpenCV

```
import cv2

# Carregar imagens
img1 = cv2.imread('livro1.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('livro2.jpg', cv2.IMREAD_GRAYSCALE)

# Inicializar detectores
sift = cv2.SIFT_create()
#surf = cv2.xfeatures2d.SURF_create()
orb = cv2.ORB_create()

# Detectar e descrever
kp1_sift, des1_sift = sift.detectAndCompute(img1, None)
kp2_sift, des2_sift = sift.detectAndCompute(img2, None)

kp1_orb, des1_orb = orb.detectAndCompute(img1, None)
kp2_orb, des2_orb = orb.detectAndCompute(img2, None)

# Correspondência (exemplo com ORB)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des1_orb, des2_orb)
matches = sorted(matches, key=lambda x: x.distance)

# Visualizar
resultado_orb = cv2.drawMatches(img1, kp1_orb, img2, kp2_orb,
matches[:50], None)
resultado_sift = cv2.drawMatches(img1, kp1_sift, img2, kp2_sift,
matches[:50], None)
cv2_imshow( resultado_orb )
cv2_imshow( resultado_sift)
```

Código-fonte 6 - SIFT *versus* ORB
Fonte: Elaborado pelo autor (2025)

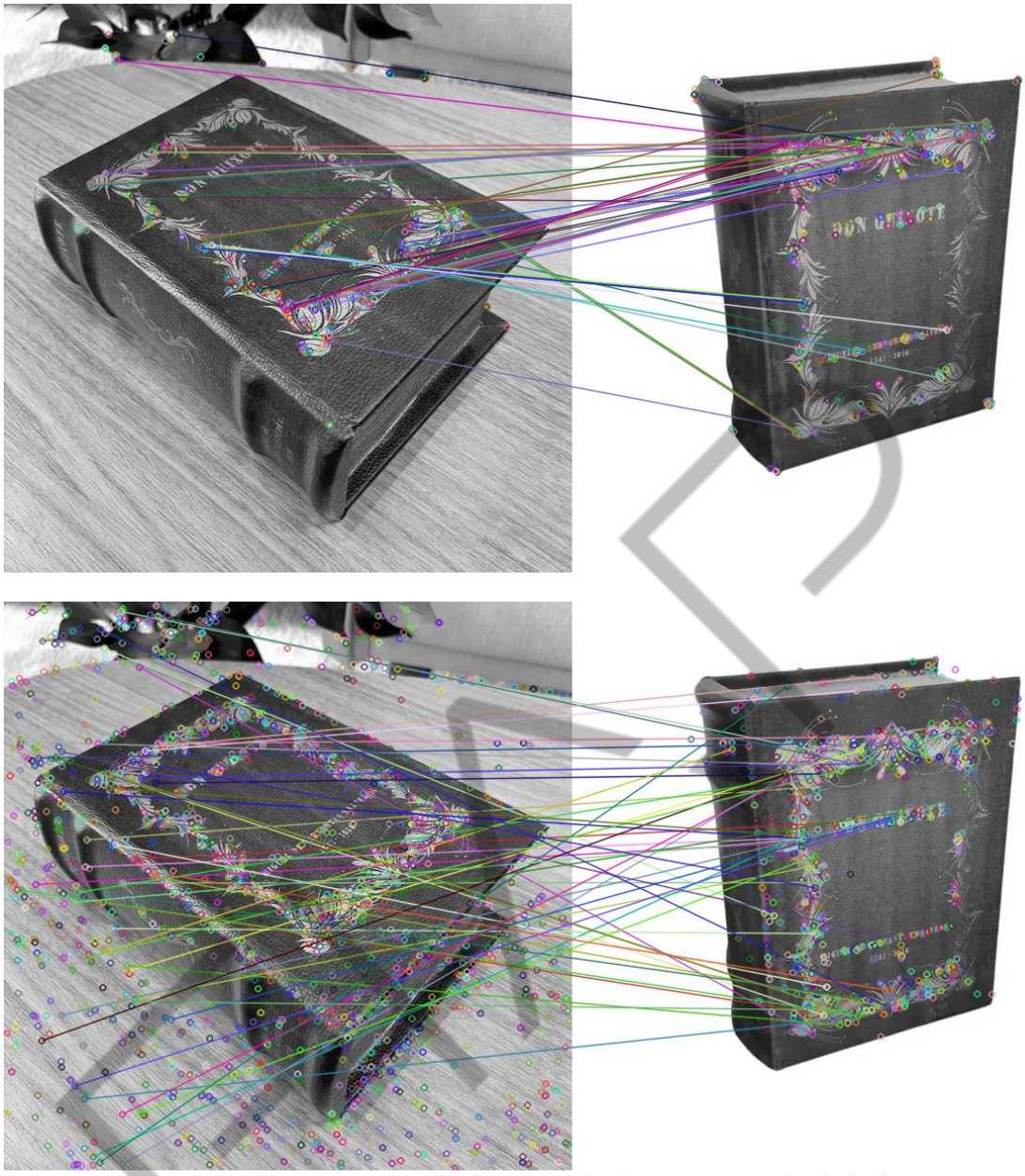


Figura 18 - Resultado ORB (cima) *versus* SIFT (embaixo)
Fonte: Elaborado pelo autor (2025)

8 DESCRITORES GLOBAIS DE CARACTERÍSTICAS (GLOBAL FEATURE DESCRIPTORS)

Quando falamos em descritores globais, estamos nos referindo a representações que caracterizam uma imagem como um todo, sem se preocupar com pontos-chave específicos como no caso dos descritores locais. Esses descritores são extremamente úteis em tarefas onde a aparência geral da imagem é mais importante do que detalhes pontuais, como na busca de imagens semelhantes em grandes bancos de dados, classificação de padrões visuais e análise de cenas.

Os dois tipos mais comuns de descritores globais são **histogramas de cores** e **descritores de textura**. Enquanto os histogramas de cores descrevem a distribuição das tonalidades na imagem, os descritores de textura capturam padrões estruturais presentes na superfície dos objetos.

8.1 Histogramas de cores

O histograma de cores é um dos descritores mais intuitivos e amplamente utilizados. Ele contabiliza a frequência de ocorrência de cada faixa de cor na imagem, criando uma assinatura que pode ser comparada entre imagens diferentes.

Uma das primeiras decisões ao trabalhar com histogramas de cores é a escolha do espaço de cores. O espaço RGB (vermelho, verde, azul) é o mais comum e direto, mas o HSV (matiz, saturação, valor) muitas vezes é preferido, pois separa o componente de cor pura (matiz) da intensidade luminosa (valor), o que pode tornar o histograma mais robusto a variações de iluminação.

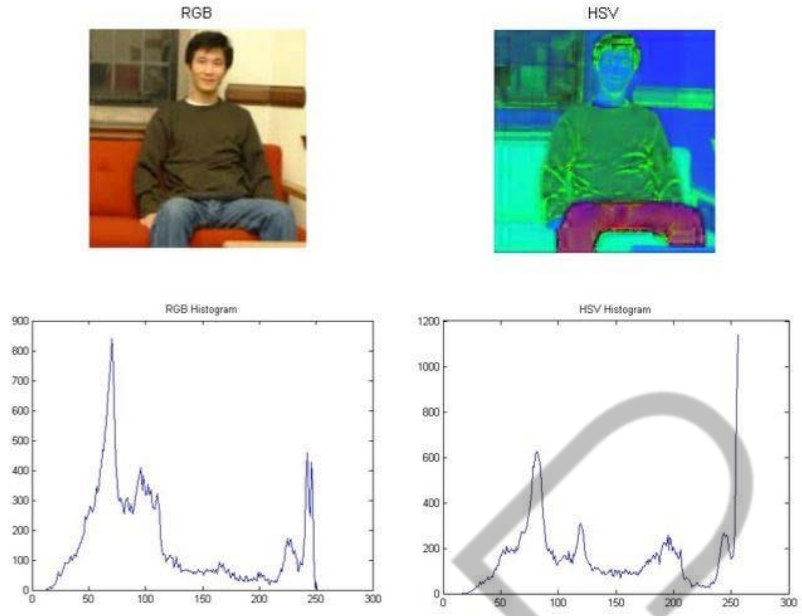


Figura 19 - Histograma RBG e HSV
Fonte: ResearchGate (2016)

Outro ponto importante é a normalização do histograma. Se duas imagens têm resoluções diferentes, a contagem absoluta de pixels não deve interferir na comparação. Por isso, normalizamos os valores para que a soma total seja 1.

A comparação entre histogramas pode ser feita com diferentes métricas, como a distância Euclidiana, a correlação de Pearson ou a distância de Bhattacharyya, sendo que esta última é muito usada em reconhecimento de padrões.

Exemplo prático: na classificação de frutas por cor, um histograma HSV pode diferenciar facilmente bananas maduras (matiz próximo ao amarelo) de bananas verdes (matiz próximo ao verde), mesmo que a iluminação varie.

8.2 Descritores de textura

Enquanto os histogramas de cores descrevem “o que” está presente na imagem em termos de tonalidades, os descritores de textura indicam como essas cores ou intensidades estão distribuídas espacialmente, capturando padrões como rugosidade, repetição e granulação.

8.2.1 Haralick Features e a Matriz de Coocorrência de Níveis de Cinza (GLCM)

As Haralick features foram propostas por Robert Haralick em 1973 e são extraídas a partir da Gray Level Co-occurrence Matrix (GLCM). Essa matriz contabiliza quantas vezes um pixel de valor de intensidade i é vizinho de um pixel de intensidade j a uma determinada distância e direção.

A partir dessa matriz, calculam-se métricas como:

- **Contraste:** mede a variação de intensidade entre pixels vizinhos.
- **Homogeneidade:** indica a uniformidade dos valores de cinza.
- **Energia:** reflete a repetição de padrões.
- **Correlação:** mede a relação linear entre os valores dos pixels.

Essas métricas são muito utilizadas em análise de superfícies, como detecção de defeitos em produtos industriais ou classificação de tipos de solo em imagens de satélite.

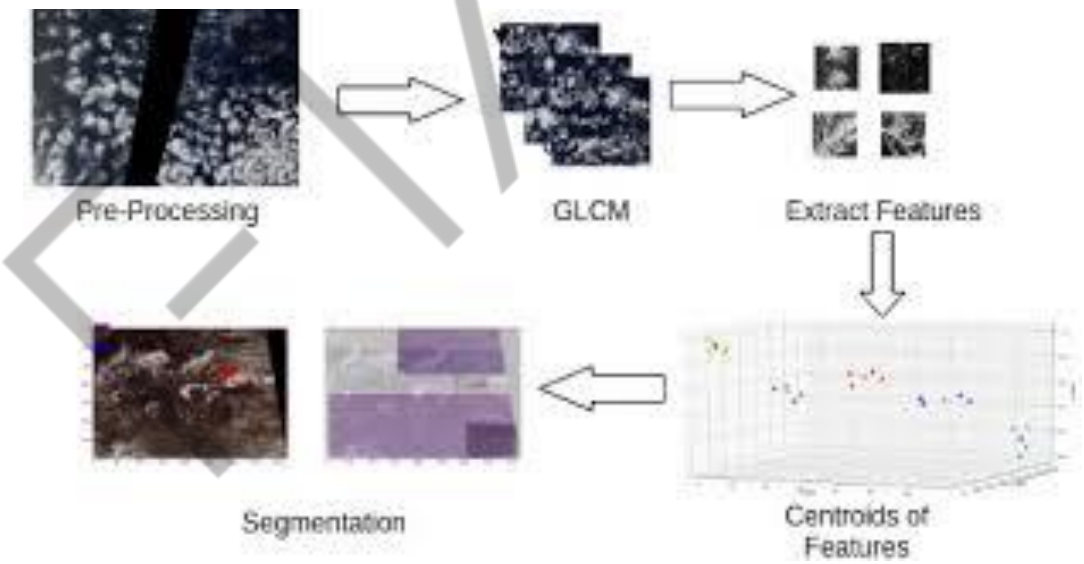


Figura 20 - GLCM
Fonte: UFF (2020)

8.2.2 LBP (Local Binary Patterns)

O Local Binary Patterns é um descritor de textura extremamente rápido e simples. Ele analisa cada pixel em relação a seus vizinhos, atribuindo valores binários

conforme o pixel central é maior ou menor que o vizinho. O resultado é um número binário que codifica o padrão local.

Por exemplo, se um pixel central tem intensidade 120 e seus vizinhos são comparados um a um, marcando 1 quando o vizinho é maior e 0 quando é menor, obtém-se um padrão binário que pode ser convertido em um número decimal.

O histograma desses números descreve a textura da imagem. O LBP é muito utilizado em reconhecimento facial e detecção de materiais, pois é robusto a variações suaves de iluminação.

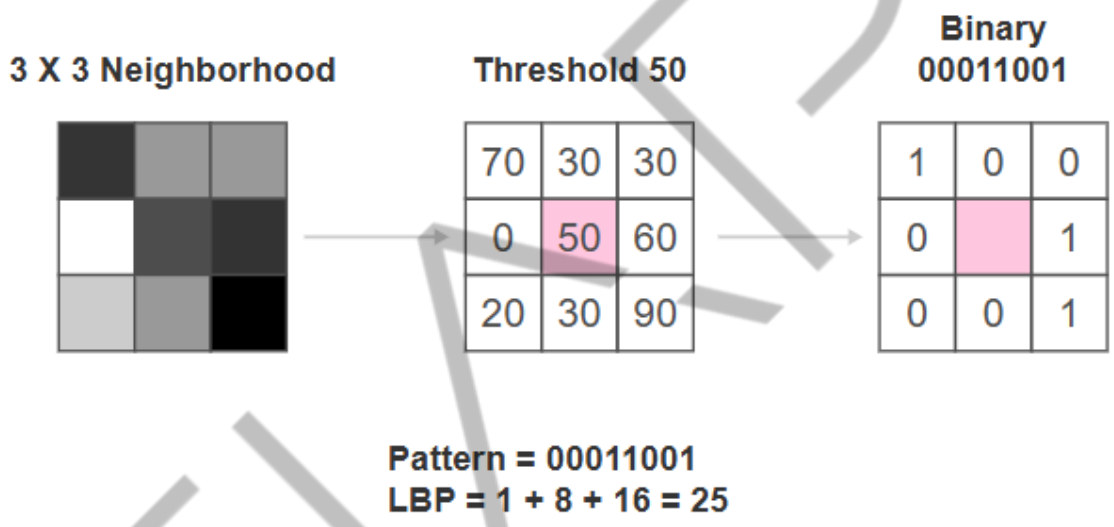


Figura 21 - LBP
Fonte: Medium (2024), adaptado por FIAP (2025)

8.3 Implementação em Python (OpenCV e Scikit-Image)

```
import cv2
import numpy as np
from skimage.feature import local_binary_pattern
from skimage.feature.texture import graycomatrix
from skimage.feature.texture import graycoprops

# Carregar imagem
img = cv2.imread('cerebro.jpg')
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Histograma de cores em HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0,
180, 0, 256])
hist = cv2.normalize(hist, hist).flatten()

# GLCM e Haralick features
glcm = graycomatrix(img_gray, distances=[1], angles=[0],
levels=256, symmetric=True, normed=True)
contrast = graycoprops(glcm, 'contrast')[0, 0]
homogeneity = graycoprops(glcm, 'homogeneity')[0, 0]
# LBP
radius = 1
n_points = 8 * radius
lbp = local_binary_pattern(img_gray, n_points, radius,
method='uniform')
(hist_lbp, _) = np.histogram(lbp.ravel(),
bins=np.arange(0, n_points + 3), range=(0, n_points + 2))
hist_lbp = hist_lbp.astype("float")
hist_lbp /= (hist_lbp.sum() + 1e-6)

print("Histograma HSV:", hist[:5])
print("Contraste (Haralick):", contrast)
print("Homogeneidade (Haralick):", homogeneity)
print("LBP (primeiros valores):", hist_lbp[:5])
```

Código-fonte 7 - Métodos de extração de features globais
Fonte: Elaborado pelo autor (2025)

9 OBJECT PROPOSALS E IDEIAS DE DETECÇÃO BASEADA EM REGIÕES

A transição entre os métodos clássicos de segmentação e as abordagens modernas de detecção de objetos passa necessariamente pelo conceito de propostas de objetos (*object proposals*). Enquanto a segmentação tradicional procura separar uma imagem em regiões coerentes com base em cor, textura ou bordas, as propostas de objetos têm como objetivo identificar regiões candidatas onde um objeto de interesse pode estar localizado, mesmo antes de se aplicar um classificador.

Essa etapa é fundamental em sistemas de detecção eficientes, pois evita a necessidade de analisar cada pixel ou cada posição da imagem de forma exaustiva, reduzindo drasticamente o custo computacional.

9.1 O Algoritmo Selective Search

O Selective Search foi um dos primeiros métodos amplamente adotados para gerar propostas de regiões de forma eficiente e é um componente essencial no funcionamento do R-CNN original (Girshick *et al*., 2014). A ideia central é combinar a robustez dos métodos de segmentação baseados em regiões com uma abordagem hierárquica de fusão.

O funcionamento pode ser descrito como uma sequência de etapas: primeiramente, a imagem é segmentada em regiões iniciais muito pequenas, geralmente utilizando o algoritmo de Felzenszwalb and Huttenlocher (2004) ou métodos semelhantes. Em seguida, essas regiões são agregadas hierarquicamente com base em similaridade de cor, textura, tamanho e proximidade espacial. Cada fusão gera novas regiões, que são adicionadas à lista de propostas.

O processo continua até que a imagem inteira seja fundida em uma única região, resultando em centenas ou milhares de propostas de objetos de diferentes tamanhos e formatos.

O Selective Search não é treinado, ele é puramente baseado em heurísticas de agrupamento, mas foi um avanço importante porque conseguiu reduzir milhões de possíveis janelas para alguns milhares sem perder muitas regiões relevantes.

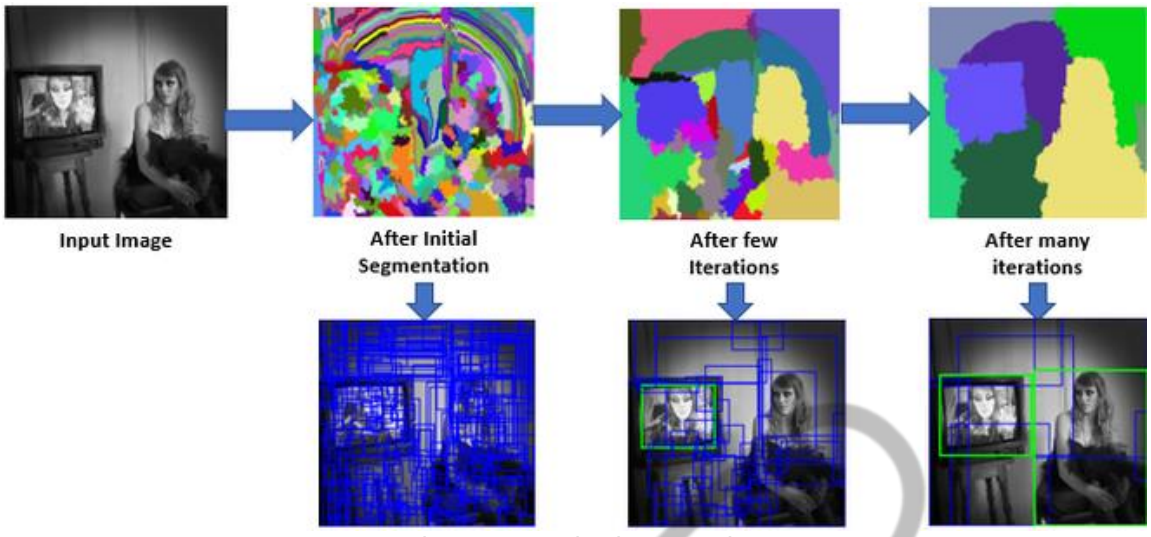


Figura 22 - Selective Search
Fonte: Medium (2025)

9.2 A Família R-CNN: Uma visão conceitual

O conceito de propostas de objetos evoluiu rapidamente para a integração com redes neurais profundas. O R-CNN (Regions with Convolutional Neural Networks) foi pioneiro nesse sentido. Sua ideia foi simples, mas revolucionária:

- Gerar propostas de objetos usando Selective Search.
- Recortar cada proposta e redimensioná-la para um tamanho fixo.
- Extrair características visuais dessas propostas usando uma rede convolucional pré-treinada (como AlexNet).
- Classificar cada proposta usando um classificador SVM e refinar as caixas com um regressor linear.

A R-CNN trouxe resultados excepcionais na época, mas tinha um problema de desempenho: processar milhares de propostas por imagem era muito lento. Isso levou ao desenvolvimento de variações como **Fast R-CNN** (que extrai features de toda a imagem de uma só vez) e **Faster R-CNN** (que substitui o Selective Search por uma Region Proposal Network, ou RPN, treinada junto com a rede principal).

Em termos conceituais, o que a família R-CNN trouxe foi a ideia de conectar segmentação aproximada com aprendizado profundo, abrindo caminho para detectores ainda mais rápidos como YOLO e SSD, que abandonam completamente a etapa explícita de propostas para operar em tempo real.

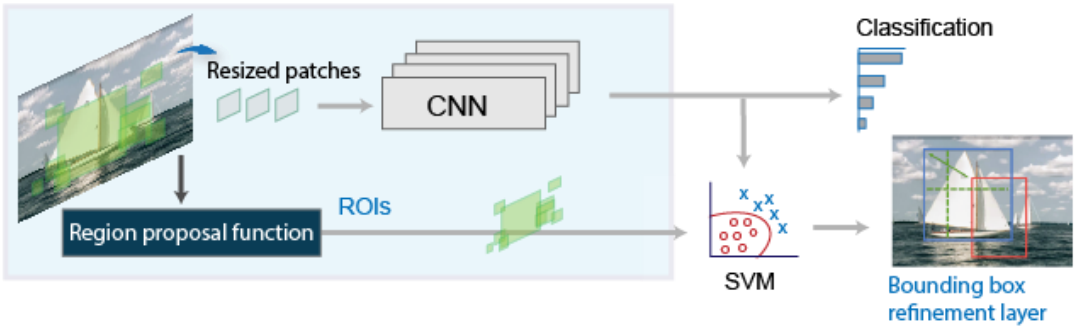


Figura 23 - R-CNN
Fonte: Medium (2020)

9.3 Conexão com aplicações do mundo real

Os métodos de propostas de objetos tiveram um impacto enorme na transição da Visão Computacional clássica para a moderna. Por exemplo, no varejo inteligente, técnicas como Selective Search foram utilizadas inicialmente para localizar produtos em prateleiras antes de treinar classificadores. Em diagnóstico médico assistido, propostas de regiões ajudaram a localizar tumores ou nódulos em exames de raio-X ou tomografia antes de uma rede convolucional analisá-los mais profundamente. Em carros autônomos, essas técnicas auxiliaram na detecção inicial de pedestres, ciclistas e veículos.

Embora atualmente os detectores como YOLO e DETR consigam localizar objetos sem a etapa explícita de propostas, entender esses métodos é essencial para compreender a evolução histórica e as bases conceituais da detecção de objetos.

CONCLUSÃO

Neste capítulo, percorremos o caminho que vai desde os fundamentos da segmentação até a aplicação de métodos modernos para identificar e descrever objetos em imagens. Compreendemos que a segmentação é muito mais do que simplesmente “recortar” partes da imagem, ela é o elo entre a captura bruta dos dados visuais e a sua interpretação, funcionando como um filtro que destaca as regiões de interesse e descarta o que é irrelevante.

Exploramos técnicas clássicas como *thresholding*, métodos baseados em regiões e detecção de bordas, entendendo suas vantagens, limitações e casos de uso. Avançamos para a extração de características, percebendo que os descritores — sejam eles locais, como SIFT, SURF e ORB, ou globais, como histogramas de cor e descritores de textura — são a ponte entre a informação visual e algoritmos de classificação ou reconhecimento.

Também vimos que, para problemas de detecção mais complexos, muitas vezes é necessário primeiro gerar propostas de objetos, usando técnicas como Selective Search e segmentação multi-escala, e que essas ideias foram essenciais para o desenvolvimento de arquiteturas modernas como a família R-CNN.

O mais importante é que agora entendemos que segmentar e extrair características não são tarefas isoladas, mas partes integradas de um pipeline de Visão Computacional. Em aplicações reais, seja em diagnóstico médico, veículos autônomos, agricultura de precisão ou varejo inteligente, a qualidade dessa etapa impacta diretamente a precisão e a eficiência do sistema como um todo.

No próximo capítulo, vamos expandir esse conhecimento, explorando como essas regiões segmentadas e características extraídas podem ser usadas em classificação e reconhecimento de objetos, conectando a percepção visual à tomada de decisão automática.

REFERÊNCIAS

ALBUKHNEFIS, Adil Lateef Mahmood. Region growing segmentation. **ResearchGate**, junho de 2024. Disponível em: <https://www.researchgate.net/figure/Region-growing-segmentation19_fig2_384734082>. Acesso em: 03 set. 2025.

ANANTA, Whisnumurty Galih. Animal Facial Expression Classification Using LBP Feature Extraction and Deep Learning Models (Combine DenseNet & ResNet Architecture). **Medium**, 17 de Janeiro de 2024. Disponível em: <http://medium.com/@whisnumurtyga/animal-facial-expression-classification-using-lbp-feature-extraction-and-deep-learning-models-89ec830b9994>. Acesso em: 03 set. 2025.

BALI, Jyoti S. a) Global thresholding with T = 0.7, b) adaptive thresholding, c) Otsu's thresholding. **ResearchGate**, abril de 2021. Disponível em: <https://www.researchgate.net/figure/a-Global-thresholding-with-T-07-b-adaptive-thresholding-c-Otsus-thresholding_fig37_351074101>. Acesso em: 03 set. 2025.

BORAH, Chinmoy. Evolution of Object Detection. **Medium**, 01 de novembro de 2020. Disponível em: <https://medium.com/analytics-vidhya/evolution-of-object-detection-582259d2aa9b>. Acesso em: 03 set. 2025.

BRADSKI, G. **The OpenCV Library**. Dr. Dobb's Journal of Software Tools, 2000.

CANNY, J. A Computational Approach to Edge Detection. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, vol. PAMI-8, no. 6, pp. 679-698, Nov. 1986.

DEBBARMA, Swapan. Example of SURF keypoints. **ResearchGate**, novembro de 2014. Disponível em: <https://www.researchgate.net/figure/Example-of-SURF-keypoints_fig10_282275014>. Acesso em: 03 set. 2025.

DEEP. Introduction to BRIEF (Binary Robust Independent Elementary Features). **Medium**, 19 de março de 2019. Disponível em: <https://medium.com/@deepanshut041/introduction-to-brief-binary-robust-independent-elementary-features-436f4a31a0e6>. Acesso em: 03 set. 2025.

DENG, J *et al.* Construction and Analysis of a Large Scale Image Ontology**. Vision Sciences Society (VSS)**, 2009.

GAUTAM, Diwakar. Split and Merge Segmentation. **ResearchGate**, julho de 2014. Disponível em: <https://www.researchgate.net/figure/Split-and-Merge-Segmentation_fig3_268195201>. Acesso em: 03 set. 2025.

GENERAL introduction to SIFT_PyOCL. **Python Hosted**, 2025. Disponível em: <https://pythonhosted.org/sift_pyocl/sift.html>. Acesso em: 03 set. 2025.

JAIN, Abhishek. RCNN — Region Based Convolutional Neural Network. **Medium**, 20 de fevereiro de 2025. Disponível em: <https://medium.com/@abhishekjainindore24/rcnn-region-based-convolutional-neural-network-c2bb37488e20>. Acesso em: 03 set. 2025.

KHULBE, Mayank. Introduction to Anomaly Detection in Images. The Good Food Economy. **Medium**, 18 de março de 2023. Disponível em: <https://medium.com/farmart-blog/anomaly-detection-in-images-using-feature-descriptors-c7124669bdfb>. Acesso em: 03 set. 2025.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. (2017-05-24). ImageNet classification with deep convolutional neural networks. **Communications of the ACM**, 24 de junho de 2017, v. 60, n. 6, p. 84–90.

LOWE, David G. Distinctive Image Features from Scale-Invariant Keypoints. **International Journal of Computer Vision**, November 2004, v. 60, n. 2, p. 91–110.

MAZET, Vincent. Histogram thresholding. **GitHub**, 20 de outubro de 2023. Disponível em: <https://vincmazet.github.io/>. Acesso em: 03 set. 2025.

METÁSTASE osteolítica occipital de carcinoma folicular da tireoide – evolução de 4 anos. **Unicamp**, 2025. Disponível em: <https://anatpat.unicamp.br/rpgmeta8.html>. Acesso em: 03 set. 2025.

QING, V.; ZHANG, Suzhen; WANG, Yuechun. Comparison between deep learning model and traditional machine learning algorithm. (a) Deep learning algorithm. (b) Traditional machine learning algorithm. **ResearchGate**, julho de 2022. Disponível em: <http://researchgate.net/figure/Comparison-between-deep-learning-model-and-traditional-machine-learning-algorithm-a_fig3_362126836>. Acesso em: 03 set. 2025.

RAHEM, Abdalrazak Tareq. Original image in RGB and HSV color space and its histogram. **ResearchGate**, fevereiro de 2016. Disponível em: <https://www.researchgate.net/figure/Original-image-in-RGB-and-HSV-color-space-and-its-histogram_fig2_299436362>. Acesso em: 03 set. 2025.

REBOUÇAS, Pedro Pedrosa Filho. Etapas de um sistema de Visão Computacional típico. **ResearchGate**, fevereiro de 2015. Disponível em: <https://www.researchgate.net/figure/Figura-1-Etapas-de-um-sistema-de-Visao-Computacional-tipico_fig4_271841344>. Acesso em: 03 set. 2025.

REDMON, Joseph *et al*. You only look once: Unified, real-time object detection. **Proceedings of the IEEE conference on computer vision and pattern recognition**, 2016.

SIMONYAN, Karen; ZISSERMAN, Andrew. Very deep convolutional networks for large-scale image recognition. **Arxiv**, v.1, setembro de 2014. Disponível em: <https://arxiv.org/abs/1409.1556>. Acesso em: 04 set. 2025.

TEXTURAS e como caracterizá-las nas Imagens. **UFF**, 2020. Disponível em: <http://www2.ic.uff.br/~aconci/temaDOIS2020-Texturas.pdf>. Acesso em: 03 set. 2025.

TIMOTHY, M. What is Feature Matching? **Roboflow**, 30 de outubro de 2024. Disponível em: <https://blog.roboflow.com/what-is-feature-matching/>. Acesso em: 03 set. 2025.

TYAGI, Mrinal. Image Segmentation. **Builtin**, 24 de março de 2025. Disponível em: <https://builtin.com/machine-learning/image-segmentation>. Acesso em: 03 set. 2025.

YANN, LeCun; BENGIO, Yoshua. Convolutional networks for images, speech, and time series. **The handbook of brain theory and neural networks**, 1995.